Information Security

From Security Goals to Cryptographic Trust

A textbook treatment of Lectures 1 to 5

Based on the lectures, TPs and quiz of Imane Fouad

October 25, 2026

Contents

How to read this book **3**

1 Foundations of Information Security **4**
1.1 Precision, Properties, Risk 4
1.2 Services and Controls 6
1.3 Actors and Attack Classes 7
1.4 Malware by Mechanism 9
1.5 Domains, Access Control, GRC 10
Chapter takeaways 11

2 The Web and Stateful Tracking **12**
2.1 The Web in Four Objects 12
2.2 Sessions and the Cookie Attachment Rule 14
2.3 Sites, Origins, Parties 16
2.4 Cross-Site Tracking versus First-Party Analytics 18
2.5 Respawning and the Other Stores 19
2.6 Cookie Syncing and Real-Time Bidding 21
2.7 Clickjacking and Framing Defences 23
Chapter takeaways 25

3 Stateless Tracking and Browser Defences **26**
3.1 Recognition Without State 26
3.2 Entropy: How Much a Feature Identifies 27
3.3 Attributes and Their Physics 28
3.4 Uniqueness, Linkability, Consistency 30
3.5 Blocking: Filter Lists and Behaviour 31
3.6 Browsers Against Tracking 32
Chapter takeaways 34

4 Symmetric Cryptography **35**
4.1 Cryptosystems, Kerckhoffs, Attack Models 35
4.2 Classical Ciphers and Their Cryptanalysis 36
4.3 XOR and the One-Time Pad 38
4.4 Confusion, Diffusion, Feistel 38
4.5 DES 39
4.6 AES 41
4.7 Modes of Operation 42
4.8 Key Exchange: Diffie–Hellman 44
Chapter takeaways 44

5 Public-Key Cryptography, Integrity and Trust **46**
5.1 Why Asymmetric 46
5.2 Number Theory for RSA 47
5.3 RSA 47
5.4 Hash Functions and Passwords 49
5.5 MACs and Digital Signatures 50
5.6 Certificates and PKI 52
5.7 Steganography 53
Chapter takeaways 54

How to read this book

Each concept in this book is presented in the same rhythm, and every section closes with a *Go deeper* box (the exact details, numbers and edge cases a demanding examiner asks about) followed by *Hard questions, answered.*

1. **Motivation and intuition.** Why do we care? What problem does the idea solve, and what picture should you hold in your head?
2. **Formal statement.** The precise definition or theorem, set in a coloured box so it is easy to find when revising.
3. **Proof.** Every claim is proved. Longer proofs are split into labelled steps, and the one idea that makes the proof work is isolated in a *Key insight* box.
4. **Interpretation and discussion.** What the result really says, where it applies, and which misreadings to avoid.
5. **Examples.** From "trivially small" to "genuinely useful."
6. **Techniques, classic problems and gotchas.** Computational shortcuts, proof patterns, exam favourites with one-line solutions, and the mistakes everyone makes once.

Margin notes carry the informal, "say it out loud" version of what is happening in the main text. Every major section opens with a *What are we trying to get?* box that states the goal in one breath before any mathematics, and closes with a summary box.

Prerequisites. Comfort with binary and hexadecimal, elementary probability, and the modular arithmetic of integers; everything else (finite fields for AES, the number theory behind RSA, Shannon entropy) is built where it is needed.

Chapter 1

Foundations of Information Security

1.1 Precision, Properties, Risk

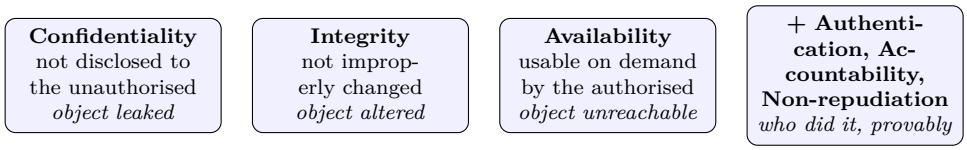
What are we trying to get?

Every later chapter is an instance of the vocabulary built here: an *asset* with a *property*, a *threat* exploiting a *vulnerability*, a *control* that preserves the property under a stated *threat model*. We want the definitions precise enough to classify any scenario, and the habit of naming all four parts of a security claim.

"Is this circle secure?" has no answer; "if I flood-fill outside the circle, does colour get inside?" does, and it depends on whether the boundary is closed. The two lessons: a security claim must name the adversary's capability before it can be judged, and the defence must be built systematically, because a freehand boundary has gaps. The course's three numbers each justify a later design choice: about 90% of attacks involve human error or behaviour (controls must bound the cost of a mistake), a breach costs $3.86M on average, and takes 191 days to discover (detection is not optional).

"Secure" is a relation: secure against what*, for* which asset*, with respect to* which property.

Definition 1.1.1 (Security properties (ISO 27001))**.** *Information security is the preservation of confidentiality, integrity and availability of information; authenticity, accountability, non-repudiation and reliability can also be involved.* **Confidentiality**: information is not made available or disclosed to unauthorised individuals, entities or processes. **Integrity**: data or resources are not improperly changed, deleted or corrupted; *data* integrity (records unaltered) versus *system* integrity (programs, configurations, processes intact). **Availability**: accessible and usable upon demand by an *authorised* entity. **Authentication**: a user or a data origin is accurately identifiable. **Accountability**: actions are traceable to those responsible. **Non-repudiation**: an actor cannot later deny an action to a third party; stronger than authentication, which only convinces the receiver.

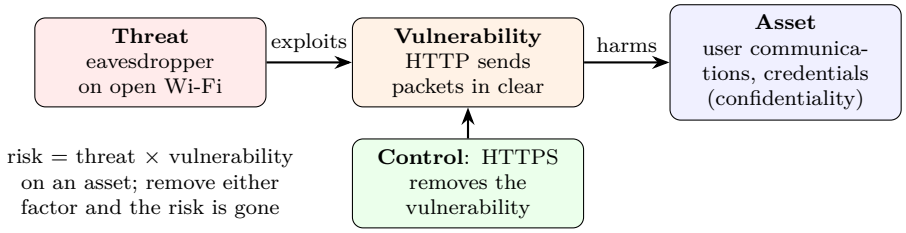


Reading the formula

To find the broken property, find the *object* the attacker touched and what the system assumed about it. Airport: Eve buys a ticket in Alice's name with a stolen card and forges a boarding pass in *her own* name; the person–pass check passes (Eve's ID, Eve's pass) and the pass–record check is skipped; a trusted document was altered: *integrity*, and no record of Eve flying exists. Targeted ads built on Alice's private browsing: *confidentiality*. The submission platform crashing before the deadline: *availability*, with no adversary at all. The properties are independent: each can fail alone, and Chapter 5 shows the cryptographic tools are equally independent (encryption gives C, a MAC gives I and authentication, a signature adds non-repudiation, nothing gives A).

Example 1.1.2 (Eve attacks Acme)**.** (1) Phishing yields a valid username and password: *confidentiality* of the credential, and *authentication* is undermined (Eve can now authenticate as the employee). (2) A buffer overflow injects code into a trusted server process that executes it as root: *system integrity*, and authentication of the actor. (3) Eve's code disables database logging: *accountability*. (4) She creates a database user by editing the table of authorised users: *integrity* of authorisation data and *authentication*. (5) Many machines flood the web server (*availability*) while she copies the database (*confidentiality*). Name properties, not techniques, and name all of them.

Definition 1.1.3 (Risk; threat model)**. Asset**: what we protect (data, systems, people). **Threat**: something that can cause damage. **Vulnerability**: an exploitable weakness. **Risk**: the potential that a threat exploits a vulnerability and harms an asset. Jim Anderson: security is "a well-informed sense of assurance that the information risks and controls are in balance". A **threat model** fixes (i) the asset, (ii) the vulnerability assumed and defended against, (iii) the attacker's capabilities and knowledge; only then can a defence be said to *prevent*, *detect*, *respond to* or *recover from* the attack. Risk treatment has four options: treat (mitigate), accept, transfer (insurance), avoid.



Key insight

Risk needs both factors: the same unpatched service is a risk on the Internet and none on an isolated machine. A control is judged *inside* its model: HTTPS answers interception on open Wi-Fi; it says nothing about phishing or a compromised laptop, which are other vulnerabilities with other attacker capabilities.

Go deeper: the details an examiner asks about

- Quiz 1 Q1–3 are one-property scenarios: reading a record without change (C), altering a result that stays accessible (I), a denial of service (A).
- "Upon demand by an authorised entity": a lockout that refuses legitimate users breaks availability without any attacker.
- Authentication has two senses in the course: user authentication (access control) and data-origin authentication (MACs, signatures); the ISO wording covers both.
- Adversaries are parameters of the model: a behavioural advertiser, Guccifer 2.0 (leaked DNC documents), and the NSA's own adversary, Snowden, the insider.

Hard questions, answered

Q. A stolen backup tape is encrypted. Which properties are affected?
A. Confidentiality preserved (strong cipher, safe key); availability of that copy lost, the organisation's availability depending on other copies; integrity of the original untouched.

Q. Why is non-repudiation not implied by authentication?
A. With a shared secret Bob is convinced Alice sent the message, but Bob could have forged it himself; a judge cannot tell. Non-repudiation needs something only Alice can produce: a signature.

Q. Threat model for a stolen password database?
A. Asset: passwords and accounts (confidentiality). Vulnerability: the verifiers are in the attacker's hands. Attacker: unlimited offline guessing with dictionaries. Control: per-user salt plus a slow hash (Chapter 5). Out of scope: observing passwords at login.

1.2 Services and Controls

What are we trying to get?

Separate what we want from how we get it, and classify every control on two axes, because the exam asks for both labels.

Definition 1.2.1 (Service versus control; two classifications)**.** A **service** (goal, property) is *what* we want: C, I, A, accountability, authentication. A **control** (mechanism) is *how*: encryption, firewalls, training; services are supported by controls. By implementation: **physical** (facility protection, guards, locks, monitoring, environmental controls, physical intrusion detection), **technical** (logical access control, cryptographic controls, security devices, user authentication, network intrusion detection, forensics), **administrative** (policies, standards, procedures and practice, personnel screening, awareness training). By function: **preventive** (prevents exploitation: encryption, a firewall rule, MFA), **detective** (warns of attempts: IDS, cameras, log review), **corrective** (repairs what was detected: restoring a known-good

image).

Scenario	Implementation	Function
MFA for company email	technical	preventive
Firewall blocks unauthorised connections	technical	preventive
Cameras record the building entrance	physical	detective
Phishing-awareness training	administrative	preventive
Backups restored after ransomware	technical	corrective
IDS alerts on suspicious traffic	technical	detective

Reading the formula

The implementation axis asks "what kind of thing is it"; the functional axis asks "when in the attack's life does it act" (before, during, after). The same word can live in two categories (a motion sensor is physical intrusion detection, a network sensor is technical); one control can serve two functions (a firewall that blocks *and* logs); and a control can be wrong for a service (encryption gives C, not A: ransomware is encryption aimed at availability). Defence in depth: controls of different categories so that one failure (a phished password) is caught by another (MFA, anomaly detection, least privilege).

Go deeper: the details an examiner asks about

- Backup is preparation; the *restore* is the corrective act; classify by the action named in the scenario.
- The three statistics map to the three functions: human error undermines prevention, 191 days is the absence of detection, \$3.86M is what correction limits.

Hard questions, answered

Q. Classify, both labels: a guard checking badges; a morning log review; lockout after five failures; a password-rotation policy; a disaster-recovery site.
A. Physical/preventive; administrative/detective; technical/preventive; administrative/preventive; technical (or physical)/corrective.

Q. One control for accountability and one for non-repudiation, with labels.
A. Protected audit logging: technical/detective. Digital signatures under a PKI: technical/preventive.

1.3 Actors and Attack Classes

What are we trying to get?

Who attacks, with what motive and resources, and what each attack class does to which property; precise enough to place a described incident in the right box.

A **hacker** is skilled; the *hat* names intent and legality: **black** (criminal, for gain or selling services on the dark web), **white** (authorised, finds vulnerabilities and helps fix them), **grey** (typically a former black hat who works with organisations but does not necessarily respect the law, e.g. tests without permission). By

motive and resources: **state-related** (very high resources; espionage or intrusion into another state's critical systems), **cybercriminals** (high resources; money), **cyber-terrorists** (destruction), **hacktivists** (ideology; defacements with a message), **script kiddies** (low capability; others' tools; amusement). A threat model needs both axes: script kiddies are stopped by patching known vulnerabilities; states buy unknown ones.

Attack class	Mechanism	Property hit
Social engineering	manipulate a person into revealing or acting; the vulnerability is the human	C, I (via the action)
DDoS	bots in a botnet flood an Internet-exposed target for hours to days; many legitimate-looking sources	A
Vulnerability exploit	weaknesses in OS, database, application (OWASP Top 10 lists web classes; Metasploit automates)	access or A
Unauthorised access	social engineering, brute force, *password cracking*: hashes already stolen, guesses tested offline	C, authentication
Privilege abuse	an insider with unnecessary privilege (no least privilege) and little logging (no accountability)	C, I, A
Ransomware	a payload: encrypt data, sell the key	A

Interpretation & discussion

Three entries carry design lessons. DDoS is hard to stop because it is *distributed*: blocking one source does nothing, so the control is capacity and upstream filtering. Password cracking is *offline*: login rate limits are irrelevant once the hash file is stolen; only the per-guess cost of the hash and the password's entropy matter. Privilege abuse has *two* enabling conditions and each is a failed principle: unnecessary privilege (least privilege) and missing logs (accountability).

Go deeper: the details an examiner asks about

- Grey hat is defined by *method*, not alternating intent; a white hat acting without authorisation is grey by this definition.
- A botnet is the product of a malware campaign and the instrument of an availability attack.

Hard questions, answered

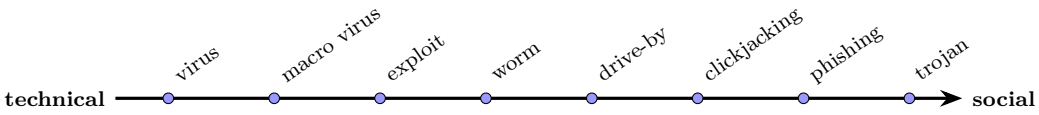
Q. Why does a six-character password survive online guessing with lockout but fall offline?
A. Online: a few tries per hour. Offline: $26^6 \approx 3 \times 10^8$ hashes computed in under a second against the stolen verifier; the only brake is the hash's per-guess cost.

Q. An admin reads customers' messages "to debug". Class and failed principles?
A. Privilege abuse; least privilege (debugging should not require content) and accountability (reads unlogged).

1.4 Malware by Mechanism

What are we trying to get?

Malware families as *mechanisms*, what each infects, how it spreads, who triggers execution, so that an incident can be classified and the right control is obvious.



what the attacker exploits: a software flaw (left) or a human decision (right); the control follows: patching and integrity monitoring vs training, procedures and interface design

Definition 1.4.1 (The families)**. Virus**: infects *programs*; modifies them to carry a copy of itself; executes when the host runs, with the host's permissions (virus = host + execution). Components: infection mechanism (the vector), trigger (a condition; a *logic bomb*), payload. **Macro virus**: document scripting; Melissa (1999) was a Word macro that accessed Outlook and mailed itself to contacts; macros now carry warnings and a separate `.xlsm` extension. **Worm**: seeks out *machines*; each infected host attacks others; spreads by network (web-application upload bugs, SSH dictionary attacks), shared media (USB autostart) or social means; strategies: *random* scanning (high traffic, noisy), *hit-list* (precompiled targets, stealthy; Stuxnet), *topological* (information on the victim), *local subnet* (behind NAT or firewall). Morris worm (1988): `fingerd` and `sendmail` bugs plus weak passwords; first conviction under the 1986 Computer Fraud and Abuse Act. **Rootkit**: with root privilege, modifies kernel code or data so that the OS lies (`ps` omits a process); *not* an infection method but stealth and continued access (a back door); needs root obtained by another attack; "runs at boot" alone is not a rootkit. **Drive-by download**: browser vulnerabilities install malware on a mere page visit (Angler kit, mid-2010s: visit, redirect, check for vulnerabilities, exploit, install). **Clickjacking**: input meant for one action performs another, by timing (a dialog appears under the "puppy" 200 ms later) or circumstance (a transparent authorisation page over a game); Chapter 2. **Phishing**: impersonate a trusted source to induce an action (click, open, reply, change a setting, transfer money); *spear* phishing targets specific victims with facts from reconnaissance; Google Docs 2017: a fake share invitation led to an OAuth consent screen, no malware, the goal was *authorisation*. **Trojan**: the victim runs it (`adorablecat.jpg.exe`, a fake ad, "install this plugin"); Zeus/Zbot banking trojan.

Key insight

Viruses infect executables to spread; worms infect machines to spread; trojans infect machines if you run them; rootkits infect kernels to avoid detection. Classical viruses are rare today because their one necessary act, modifying a binary, is cheaply prevented (no write access to program files) and detected (tracking binary hashes). Ransomware is a payload, not a vector.

Example 1.4.2 (Guess the type)**.** A: an outdated Windows service is exploited, the malware scans the LAN automatically, reaches 200,000 machines in 150 countries, encrypts files and demands Bitcoin: a *worm* with a *ransomware* payload. B: a compromised accounting-software update; spreads by a network exploit *and* stolen credentials; shows a ransom note but paying never recovers the files: *supply-chain* delivery, *worm* propagation, a *wiper* disguised as ransomware. Classify by mechanism, not by the note on the screen.

Technique: Phishing defences

Users: skepticism towards all communications; limit what you publish (the attacker's background material); inspect links; reach known services yourself. Organisations: training; chain-of-command procedures so that an email is never a valid channel for critical requests; email link screening, which shows all outgoing clicks, reveals trends and black-holes links at the forwarding stage (a detective control that becomes corrective).

Go deeper: the details an examiner asks about

- Rootkit detection must come from outside the lying kernel: boot clean media, compare against known-good images.
- Hit-list = targeted and stealthy; random = fast and loud; the slides pair the words deliberately.
- Consent-screen phishing defeats "inspect the link"; organisation-side controls matter.

Hard questions, answered

Q. Hashing binaries stops viruses but not worms. Why?
A. A virus must modify executables, so hashes change; a worm arrives as a new process through a network exploit and touches no existing program. Worm controls: patching, segmentation, credential hygiene.

Q. A program hides its files, survives reboot, and came as an email attachment the user opened. Vectors and categories?
A. Phishing delivery of a trojan; persistence is ordinary; hiding files by altering OS answers is a rootkit component, which needed root (an exploit, or the user running it as administrator).

1.5 Domains, Access Control, GRC

Technical domains: network security (traffic, firewalls), web security (applications, sessions, input), cryptography (encryption, hashing, signatures, PKI), access control (authentication, authorisation, least privilege), endpoint security (operating systems), cloud security (monitoring, detection, response). Governance domains: GRC and privacy. Chapters 2 and 3 are the privacy domain; Chapters 4 and 5 are cryptography.

Definition 1.5.1 (IAAA; GRC)**.** Access control decides who or what may access a resource and what they may do: *identification* (claim an iden-

tity), *authentication* (prove it), *authorisation* (what the identity may do), *accountability* (log and audit). Each fails separately: a badge reader identifies the badge, not the holder (add a PIN); a logged-in user reaching `/admin` is an authorisation failure; a shared account defeats accountability. **GRC**: governance (direction and oversight, policies, roles, strategy), risk (identify, assess likelihood and impact, treat/accept/transfer/avoid, monitor), compliance (laws, standards, contracts, evidence and audits). Web model: `subdomain.mysite.com/folder/page?id=5`; GET fetches pages and resources, POST submits forms and logins.

Go deeper: the details an examiner asks about

- Least privilege lives in authorisation; privilege abuse is its failure.
- References to cite: Whitman & Mattord; Stallings & Brown; Anderson's *Security Engineering*; ISO/IEC 27001 and 27002; NIST; OWASP.

Hard questions, answered

Q. Map a TLS connection to a bank onto IAAA.
A. Identification: the server name; authentication: the certificate chain (server) and password/MFA (user); authorisation: the account's permitted operations; accountability: transaction logs tied to the session.

Q. Cyber-insurance and refusing to store card numbers are which treatments?
A. Transfer; avoid.

Chapter takeaways

Summary: Foundations

- Asset, property, adversary, control: all four or it is not a claim. Properties are independent; find the tampered object.
- Risk = threat × vulnerability on an asset; model before defending; treat, accept, transfer, avoid.
- Service = what, control = how; label controls by implementation *and* function.
- Malware by mechanism: virus (programs), worm (machines), trojan (the user runs it), rootkit (the kernel lies); the social end of the axis exploits the person.
- IAAA for access control; GRC for governance.

Chapter 2

The Web and Stateful Tracking

2.1 The Web in Four Objects

What are we trying to get?

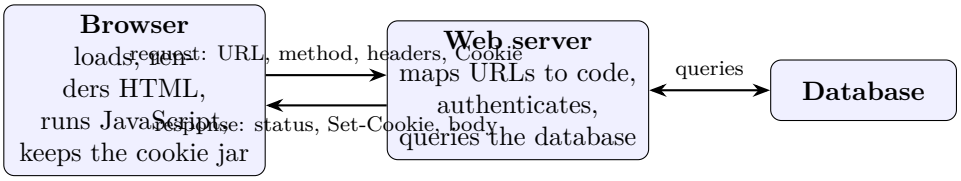
Four objects carry every argument in this chapter: the URL (what is requested), HTTP (how, and why the server forgets), HTML and JavaScript (what the browser does with the answer), and the browser's own view of the page. We also fix what "privacy" means so that "this mechanism threatens privacy" is a precise claim.

Definition 2.1.1 (Privacy)**.** Privacy concerns how information about people is *collected, used, shared and retained*, and a person's ability to *understand* what happens to their information, to *influence* how others use it, and to *avoid* unwanted observation or interference.

Reading the formula

Two halves: a data life cycle (four verbs where a tracker acts) and the person's agency (three verbs a tracker defeats). Invisible tracking defeats *understand*; tracking with no opt-out defeats *influence*; tracking that survives cookie deletion defeats *avoid*. The cases the course cites: Cambridge Analytica (Christopher Wylie, 18 March 2018: "we exploited Facebook to harvest millions of people's profiles . . . and target their inner demons"); Google's $392 million settlement with 40 US states (November 2022) over misleading location tracking.

Definition 2.1.2 (URL, HTTP, HTML)**.** In `https://informationsecurity.com:443/courses/web-security?week=5&topic=xss` the parts are the *protocol*, the *hostname* (resolved to an IP address by DNS), the *port*, the *path* and the *query string*. HTTP is an application-layer, client–server, request–response protocol: a request names a URL and carries a method (GET, POST), headers and an optional body; a response carries a status code and the resource or an error. HTTP is **stateless**: the server keeps nothing once the TCP connection closes. HTML describes structure (text, images, links, forms); JavaScript supplies logic.



The browser loads a window or frame, renders it (parsing HTML, executing scripts), and then reacts to events: user actions (`onclick`, `onmouseover`), rendering events (`onload`, `onbeforeunload`, `onerror`), timers (`setTimeout`). JavaScript reads the current URL through `location`: `href` is the complete URL including the fragment, `host` is hostname plus port, `hostname` the host alone, `pathname` the path, `search` the query. History, for completeness: Bush's Memex (1945, non-linear navigation), Berners-Lee's ENQUIRE (1980, CERN, local only), the 1989 proposal *Information Management: A Proposal* that became the WWW.

Key insight

Two facts drive everything that follows. (1) *Statelessness* means an application (a shopping cart) cannot exist unless state is added on top of HTTP; trackers reuse that added state, so tracking is the web's own design pointed at another goal. (2) *A script runs with the authority of the page that includes it*, not of the server that served it: `analytics.js` fetched from `metrics.test` but included by `shop.test` executes as `shop.test`'s code and sees `shop.test`'s cookies.

Go deeper: the details an examiner asks about

- The fragment (`#...`) is in `location.href` but is never sent in HTTP; a script must read and transmit it.
- GET fetches (a link click is a GET); POST submits forms and logins. `SameSite=Lax` (§2.2) distinguishes them.
- Stateless ≠ anonymous: every request carries the IP and headers (User-Agent, Accept-Language) that recognise a browser without state (Chapter 3).

Hard questions, answered

Q. `https://a.example/p#secret` includes `<script src="https://t.example/x.js">`. What does `t.example` learn from the request for `x.js`, and what could the script learn afterwards?
A. From the request: t.example's own cookies, the Referer `https://a.example/p` (no fragment), IP and headers. Afterwards the script, running as `a.example`, can read `location.href` including `#secret` and a.example's non-HttpOnly cookies, and send them to t.example explicitly.

2.2 Sessions and the Cookie Attachment Rule

What are we trying to get?

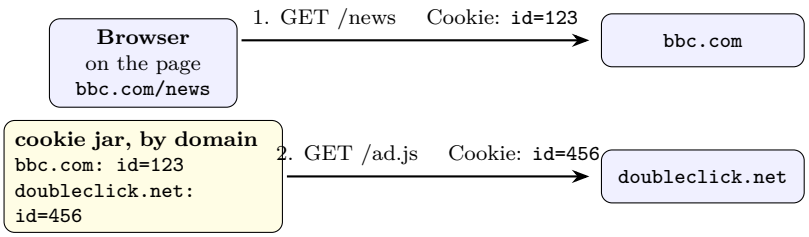
We want the two ways a stateless server can recognise a returning browser, why the first leaks, and the exact semantics of the second, cookies, because every tracking mechanism and every defence in this chapter is a consequence of one rule.

Option 1: the identifier in the URL

Generate a random token on the first visit and append it to every link: `?sessionid=8F4A92C1`. It works and it leaks: the URL is copied into bookmarks and messages, written into server logs, and transmitted in the `Referer` header of every request the page triggers, so an image loaded from a third-party server receives the full URL of the embedding page, session included.

Option 2: cookies (1994)

Definition 2.2.1 (Cookie; the attachment rule)**.** A server stores a name–value pair in the browser with a `Set-Cookie` response header. The browser persists it and *sends it back in a* `Cookie` *request header on every subsequent request to that domain, regardless of which site initiated the request.* Without `Expires`/`Max-Age` the cookie is a *session cookie*, deleted when the browser closes; with one it is *persistent*. Ending a session server-side means deleting the cookie.



Request 2 was caused by the bbc page (it embeds the script), but the jar only looks at the *destination* domain and attaches doubleclick's cookie.

Reading the formula

Both clauses of the rule are load-bearing. *To that domain*: no site can read or receive another site's cookie, so identifiers must be smuggled through URLs to cross sites (§2.6). *Regardless of initiating site*: when `bbc.com` embeds a script from `doubleclick.net`, the request to doubleclick carries doubleclick's cookie even though the user is "on" bbc; that is third-party tracking, and it is also cross-site request forgery. The jar is indexed by destination domain and does not know which page caused the request; the server cannot tell either unless it looks at the `Referer` (or modern `Sec-Fetch-*`) headers. `SameSite`, added in 2016, qualifies exactly this clause.

Attributes

Definition 2.2.2 (Scope, lifetime, security attributes)**.** `Domain`: default is the current host only; set to a parent domain (`Domain=example.com` from `websec.example.com`) to cover it and all its subdomains; cannot name an unrelated domain or a public suffix. `Path`: that path and its sub-paths. `Expires`/`Max-Age`: lifetime. `HttpOnly`: not readable through `document.cookie`; still *sent*. `Secure`: sent only over HTTPS and settable only from an HTTPS response. `SameSite=Strict`: never attached to cross-site requests. `SameSite=Lax` (browser default when absent): attached to cross-site requests only if they are *top-level navigations* with a *safe method* (GET, HEAD, OPTIONS, TRACE per RFC 7231). `SameSite=None`: attached to all cross-site requests; requires `Secure` (Chrome rejects None without Secure; Firefox and Edge warn).

Reading the formula

Each security attribute closes one door. `HttpOnly` closes *script theft*: an injected script cannot read the session cookie; it does nothing about the cookie being sent, so it does not touch CSRF or tracking. `Secure` closes *network theft*. `SameSite` closes the *cross-site attachment* door of the rule: Strict shuts it (a link from another site arrives logged out); Lax leaves it open only for an ordinary link click (top-level, GET), the case that breaks the web least; None keeps the 1994 behaviour, which a third-party tracker needs and which therefore must be HTTPS-only.

Technique: Is cookie c attached to a request from page P to destination D?

1. Is D the cookie's domain, or a subdomain of it when `Domain` was set? If not: never.
2. Does the request path start with the cookie's path?
3. If `Secure`: is the request HTTPS?
4. Do P and D have the same registrable domain (§2.3)? If yes: attached. If no (cross-site): Strict ⇒ no; Lax ⇒ only if top-level navigation *and* safe method; None ⇒ yes.

`HttpOnly` never enters: it only governs `document.cookie`.

Worked example: Quiz 1, question 4

`https://shop.example.test` sets `aid=A72; Domain=example.test; Path=/; Secure; HttpOnly; SameSite=Lax`.

A From the shop, load `https://cdn.example.test/banner.png`: `cdn.example.test` is under `example.test`; HTTPS; same site, so no cross-site question: *attached*.

B The same over `http://`: `Secure` fails: *not attached*.

C From `https://social.test`, Alice *clicks a link* to `https://shop.example.test/profile`: cross-site, but a top-level

GET navigation, which Lax permits: *attached*.

D `https://social.test` *embeds* `https://shop.example.test/tracker.png`: cross-site and not top-level: *not attached*.

Answer: A and C. `HttpOnly` was a distractor.

Gotcha

Lax has *two* conditions and both must hold: a cross-site iframe or image fails "top-level"; a cross-site POST form is top-level but fails "safe method". Only a plain link passes. Students who remember "Lax allows navigation" and forget "GET only" answer CSRF questions wrong.

Go deeper: the details an examiner asks about

- `Set-Cookie: test=1; Domain=.example.com; Secure; HttpOnly; SameSite=None`: sent on every HTTPS request to example.com and its subdomains, cross-site included, invisible to JavaScript, and still a *session* cookie (no expiry). Without `Secure`, Chrome drops it.
- Scope is by *name*: in the TP, `lab.test:8000` and `127.0.0.1:8000` are different jars for the same Flask server, which is why the hosts file is edited.
- Trackers need `None; Secure`; banks choose Strict (no CSRF at all) or Lax (inbound links arrive logged in).

Hard questions, answered

Q. A bank cookie is `HttpOnly; Secure` with no SameSite. An attacker page auto-POSTs a transfer form to the bank. Is the cookie sent?
A. Under the Lax default: no, a cross-site POST fails the safe-method condition. Treated as None (old browsers): yes, and the transfer runs as the victim. HttpOnly and Secure are irrelevant to attachment.

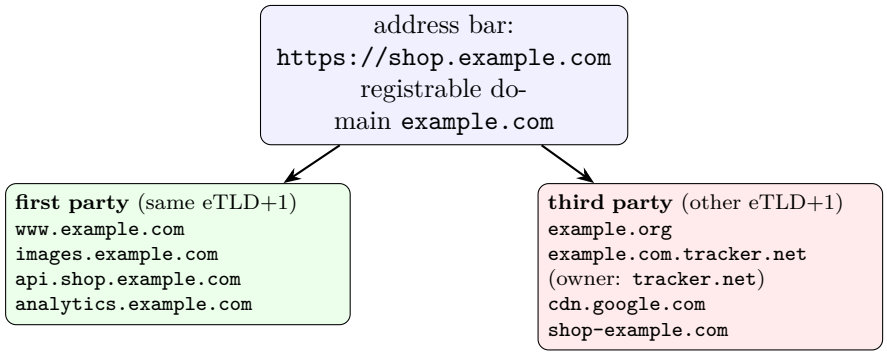
Q. Can `a.example.com` set a cookie that `b.example.com` receives? One that `example.org` receives?
A. Yes, with `Domain=example.com`; no, a different registrable domain.

Q. Does `HttpOnly` stop a tracker?
A. No: the tracker does not read its cookie with script; the browser *sends* it. HttpOnly only blocks `document.cookie`.

2.3 Sites, Origins, Parties

Definition 2.3.1 (Registrable domain; first and third party; origin)**.** The **effective top-level domain** (eTLD) is the longest public suffix a hostname ends with (`com`, `co.uk`, `github.io`); the **registrable domain** (eTLD+1) adds one label: the unit a single owner registered. A request is **first-party** when its destination has the registrable domain shown in the address bar, **third-party** otherwise; cookies are first- or third-party accordingly. An **origin** is (scheme, host, port); the **same-origin policy** lets a script read documents, storage

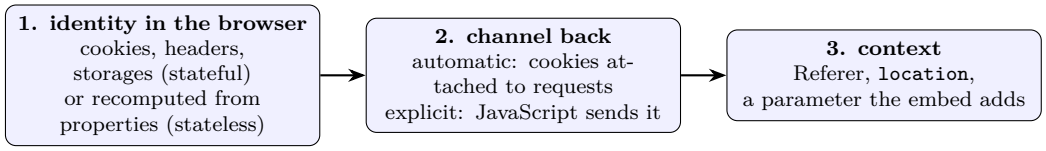
and `document.cookie` only of its own origin. Subdomains are same-site but cross-origin.



Reading the formula

Why registrable domains? Because they are the unit of *ownership*, read from the right (the `facebook.mobile.com` lesson of Chapter 1). *Site* governs cookie *attachment* (SameSite, partitioning); *origin* governs script *access*. The same-origin policy never prevents the browser from sending a cookie (Quiz 1 Q6C is false); it prevents `bbc.com`'s script from reading doubleclick's cookie.

The three capabilities of a tracker



Remove (1) and every visit is a stranger; remove (2) and the tracker never hears; remove (3) and it knows "123 is back" but not where. Chapter 3 replaces (1) by stateless recognition.

Hard questions, answered

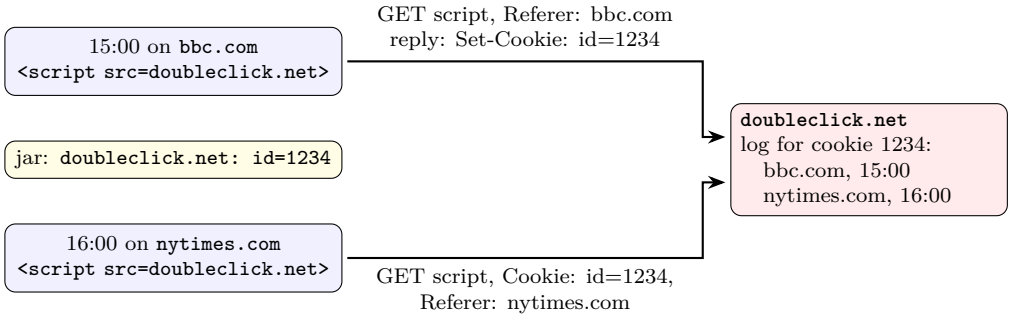
Q. Visiting `https://news.bbc.co.uk`, classify `static.bbc.co.uk`, `bbc.com`, `bbc.co.uk.cdn-edge.net`.
A. First (registrable domain `bbc.co.uk`); third (`bbc.com` is another registrable domain); third (owner `cdn-edge.net`).

Q. Can a script on `a.example.com` read `b.example.com`'s cookies? Are `Domain=example.com` cookies sent to both?
A. No (cross-origin: SOP blocks access); yes (same-site; Domain covers both). Access and attachment are different rules.

2.4 Cross-Site Tracking versus First-Party Analytics

What are we trying to get?

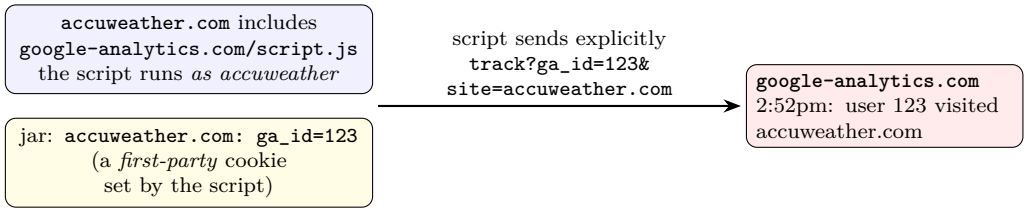
We want to derive, from the attachment rule alone, how one third-party cookie links visits across unrelated sites, and to see why the similar-looking first-party analytics cannot: the difference is under which domain the cookie is stored.



Theorem 2.4.1 (One third-party cookie)**.** *A tracker embedded on a set of sites $\mathcal{S}$ with a persistent cookie learns, per browser, the timed sequence of visits (site and page) to every site in $\mathcal{S}$, with no cooperation from the sites.*

Proof. Each visit to $s \in \mathcal{S}$ triggers a request to the tracker ("regardless of initiating site"); the request carries the tracker's cookie ("to that domain"), the same value on every site because it is stored under the tracker's domain, not under s; the Referer or an embedding parameter supplies s and the page. Grouping the log by cookie value yields each browser's history. □

First-party analytics



On nytimes.com the same script creates a *different* cookie under nytimes.com; Google receives two identifiers it cannot link.

The script file comes from Google but executes as accuweather's code (key idea of §2.1), so the cookie it creates belongs to accuweather. The browser will never attach accuweather's cookie to a request destined for google-analytics, so the script must put the value into the URL itself. Analytics therefore measures repeat visits *within* a site; it cannot collect history *across* sites.

Key insight

Tracker-domain cookie: one identifier everywhere, attached automatically. Publisher-domain cookie: one identifier per publisher, sent explicitly. "Third-party cookies blocked" blocks cookies set in a third-party context; it does not stop a third-party *script* from setting first-party cookies, which is the loophole analytics lives in.

Worked example: Quiz 1, questions 5, 6, 7

Q5 (shop stores non-HttpOnly first-party `prefs`; page loads `metrics.test/analytics.js` and an iframe from `tracker.test`): the script runs in the shop's origin and can read `prefs` (A true) and send it explicitly (D true); the iframe is another origin, SOP forbids reading (B false); the request for `analytics.js` carries metrics.test's cookies, not the shop's (C false). *Q6* (pixel from `tracker.test`, cookie `tid; SameSite=None; Secure`, third-party cookies allowed): both publishers' requests carry the same `tid` (A true, Theorem 2.4.1); the tracker links the visits (D true); publisher JavaScript cannot read it (B false); SOP does not govern sending (C false). *Q7* (third-party cookies blocked; same script on `news.test` and `shop.test`; `aid=N31` and `aid=S84`): first-party identifiers (A true); never sent automatically to metrics.test (B false); same script does not mean same identifier (C false); the script can read and send the current one (D true). Merging N31 and S84 needs an external link (a login on both, a fingerprint).

Go deeper: the details an examiner asks about

- Publishers set `HttpOnly` on their session cookie partly against the third-party scripts they include themselves.
- Citations: Roesner et al. (NSDI 2012) classified tracking behaviours; Englehardt & Narayanan (CCS 2016, OpenWPM, one million sites) measured third-party presence.

Hard questions, answered

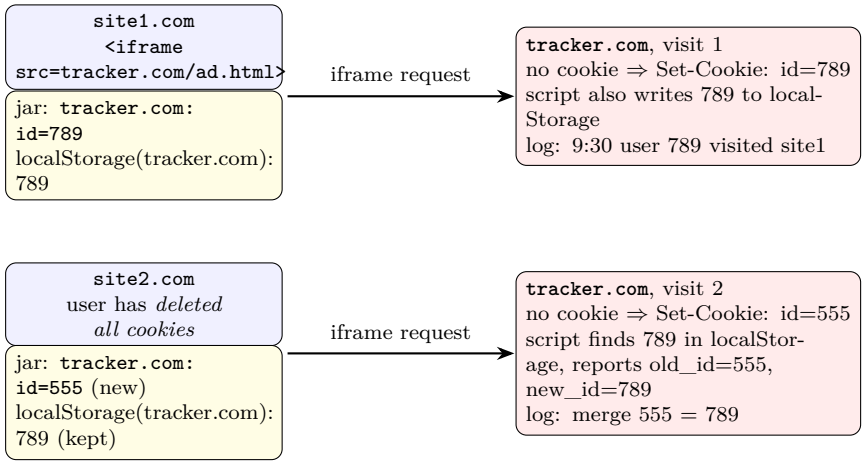
Q. Minimal change turning an analytics vendor into a cross-site tracker?
A. Have the script load a resource from the vendor's own domain that sets a vendor-domain cookie with `SameSite=None; Secure`; by Theorem 2.4.1 it links every site that includes the script. Under the Lax default the embedded request would not carry it.

2.5 Respawning and the Other Stores

What are we trying to get?

Deleting cookies is not enough: the browser has other persistent stores, and a tracker can restore the cookie from any of them. We want each mechanism, and the discipline of reading an experiment ("consistent with" is not "proves").

Definition 2.5.1 (Respawning)**. Cookie respawning** recreates a deleted cookie from a copy held elsewhere (a *zombie* cookie): HTML5 `localStorage` (string key–value pairs per origin, no expiry, across sessions), Flash local shared objects (per machine user, so across sessions *and* across browsers), HTTP cache headers `ETag` and `Last-Modified`. Evercookie (Samy Kamkar) combines many stores. Each store can also track *without* any cookie.



The iframe's script runs in `tracker.com`'s origin on every embedding site, so `tracker.com`'s localStorage is reachable from all of them: a cross-site store that cookie deletion does not touch. The user sees a new cookie value and is still the same person to the tracker.

The ETag

`ETag` is a caching mechanism: the first response to `GET A.com/i.js` carries `ETag: "x234dff"`; the browser caches the body with the tag and next time sends `If-None-Match: "x234dff"`, to which the server may answer `304 Not Modified`. Nothing forces the tag to describe the content; a tracker can make it a user identifier, and the browser will echo it on every request for that resource, even in private browsing with cookies blocked, as long as the cache lives. KissMetrics (lawsuit, August 2011) set `_km_cid=Z9iG...` as a cookie *and* as the ETag with the same value; a later request in private mode with cookies blocked still sent `If-None-Match: "Z9iG..."`. `Last-Modified` works the same way through `If-Modified-Since`.

Interpretation & discussion

Each store has its own scope and its own clearing action: cookies by domain ("clear cookies"), localStorage by origin ("clear site data"), the cache ("clear cache"); Flash objects were cleared by none of them. A privacy action is sufficient only if it clears every store the tracker might have used, which is the logic of Quiz 1 Q12.

Worked example: Quiz 1, questions 10 to 12

C = cookie, L = localStorage, E/F = ETag / fingerprint, P = server-side profile.

Test	Action before visiting	C	L	E / F	P
1	nothing	1234	1234	1234 / 88	3
2	delete the cookie	1234	1234	1234 / 88	3
3	delete cookie + localStorage	1234	new	1234 / 88	3
4	clear cookies, storage and cache	5678	5678	5678 / 88	3
5	different device	9012	9012	9012 / 42	9

Test 2: the cookie came back while localStorage still held 1234: *consistent*

with restoration from localStorage (Q10 A); it does not *prove* fingerprinting (B false), a stateful store explains it. Test 3: localStorage was wiped ("new"), yet the cookie is 1234 and the ETag still holds 1234: consistent with ETag restoration (C); the profile stayed 3, so deletion did not break the link (D false). Test 4: every store cleared, new identifiers 5678 everywhere (so the stores *were* cleared, Q11 C false), but fingerprint 88 and profile 3 persist: consistent with *stateless* relinking (B); a fingerprint identifies a browser, not a named person (A false); other browsers may share 88, so linking can produce false positives (D). Q12: the sufficient action against both respawns is to clear cookies, local storage and the relevant cache *together* (A); the other options leave a source intact.

Hard questions, answered

Q. Design a respawn that survives "clear site data" but not "clear cache".
A. Put the identifier in the `ETag` of a resource the page always loads; it returns in `If-None-Match`. Site-data clearing removes cookies and storage, not the HTTP cache.

Q. Why could Flash respawn a cookie in a *different* browser?
A. The plugin's storage was per machine user, outside any browser profile; Chrome and Firefox shared it.

Q. Everything cleared, the profile still continues. The only remaining explanation?
A. A stateless identifier recomputed from the browser's properties: Chapter 3.

2.6 Cookie Syncing and Real-Time Bidding

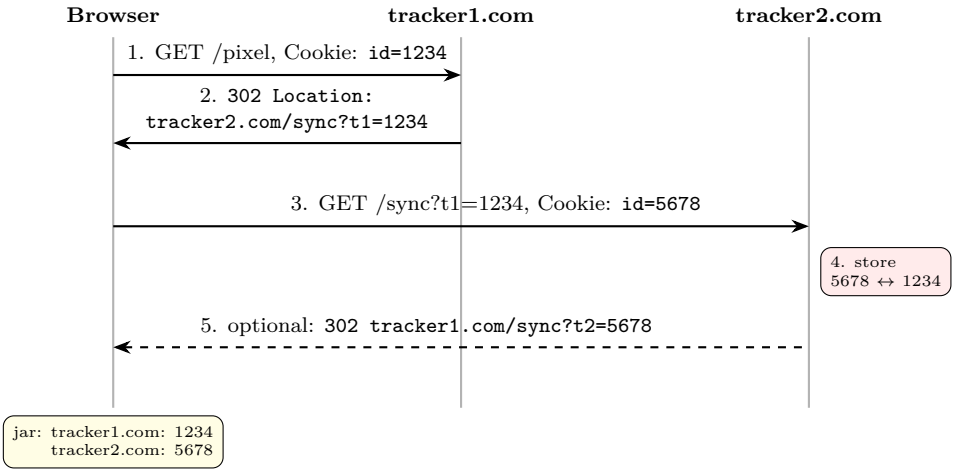
What are we trying to get?

Theorem 2.4.1 is bounded by the sites a tracker is embedded on, and two trackers hold two unrelated identifiers that neither can read. We want the protocol trick that joins them, and the advertising auction in which the trick is routine and personal data is broadcast.

The limit

Doubleclick is embedded on bbc and nytimes; adnxs on bbc and jumia. Doubleclick's log reads "bbc 15:00, nytimes 16:00"; adnxs's reads "bbc 15:00, jumia 17:00". Neither sees the union, and each has its own cookie for the browser that the other cannot receive ("to that domain"). Two partial profiles exist that nobody can join.

Definition 2.6.1 (Cookie syncing)**. Cookie syncing** is the process by which two different trackers link the identifiers they have each assigned to the same browser.

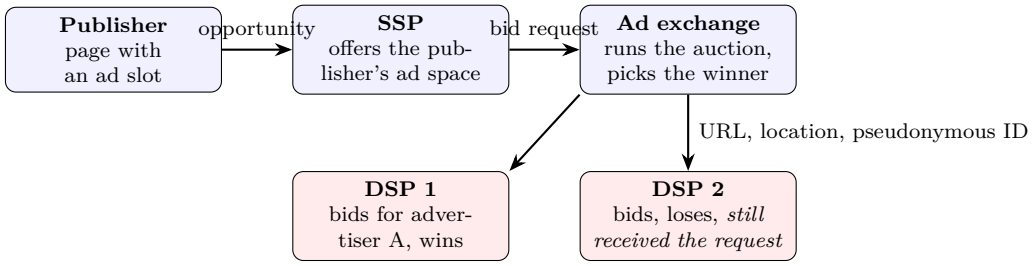


Step by step. (1) A publisher page embeds a pixel from tracker 1; the request carries tracker 1's cookie, `id=1234`. (2) Tracker 1 does not answer with an image; it answers with a redirect whose `Location` URL contains its own identifier for this browser. (3) The browser follows the redirect automatically; the new request is to tracker 2's domain, so by the attachment rule it carries *tracker 2's* cookie, `id=5678`, and the URL still carries `t1=1234`. (4) Tracker 2 now sees both values in one request and records the pair. (5) Optionally it redirects back so tracker 1 learns the mapping too. From then on both trackers can exchange data about this browser under either identifier; tracker 1's demographics (age 25, female) join tracker 2's contact data (email, address), and each tracker's coverage becomes the union of their sites.

Key insight

Nothing illegal happened at the browser level: it followed a redirect, as it must, and attached each domain's cookie to the request for that domain, as it must. The identifier crossed the domain boundary *inside the URL*, the one place the attachment rule does not police. Neither tracker ever read the other's cookie.

Real-time bidding



Real-time bidding transacts display ads in real time. The **SSP** (supply-side platform) helps publishers offer and sell ad space and forwards each opportunity to the exchange; the **ad exchange** runs the auction, distributes the bid request and selects the winner; a **DSP** (demand-side platform) evaluates opportunities and bids automatically for the users its advertisers want. The bid request (page URL, approximate location, pseudonymous identifier) is *broadcast to all DSPs*, so participating in the auction, not winning it, is what leaks. Inside RTB an SSP syncs its cookie with a DSP by exactly the redirect above, so the DSP recognises in the request a user it has profiled. Only part of the process is visible in the

browser; what each DSP retains cannot be observed from the client, which is why transparency studies are hard and regulators must audit companies.

Worked example: Quiz 1, questions 8, 9 and 16

Q8: `GET tracker-a.test/sync, Cookie: aid=A42` → `302 Location: tracker-b.test/sync?aid=A42` → `GET tracker-b.test/sync?aid=A42, Cookie: bid=B71`. Tracker B can associate A42 with B71 (A true); A42 arrives as a URL parameter, not as B's cookie (B false); A passed its identifier through the redirected URL (C true); the trackers need not share an origin, that is the point (D false). *Q9*: the SSP helps the publisher sell (A true); the losing DSP-X still received the request (B true); not only the winner (C false); the pseudonymous ID is stable and *enables* linking (D false). *Q16* asks for the drawing above: browser in the middle with both jars; request 1 carrying 1234; the 302 with 1234 in the URL; request 2 carrying 5678 plus the parameter; the mapping table at tracker 2; the optional return redirect.

Go deeper: the details an examiner asks about

- Syncing merges *coverage* and *attributes*, and moves identifiers into URLs, where they leak through Referer headers and logs.
- Andrew Lewis: "if you're not paying for the product, you are the product."

Hard questions, answered

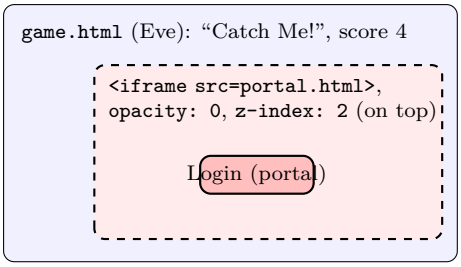
Q. Firefox gives each top-level site its own cookie jar (Chapter 3). Does that stop a sync between two trackers embedded on the *same* publisher?
A. No: on that publisher both trackers have cookies in that site's jar and the redirect links them there. Partitioning stops the *reuse* of one tracker's cookie across publishers.

Q. Why does syncing need a redirect rather than a script call?
A. Only a request *to tracker 2's domain* carries tracker 2's cookie; the redirect makes the browser issue such a request while carrying tracker 1's identifier in the URL. No script on the page can read either cookie across origins.

2.7 Clickjacking and Framing Defences

What are we trying to get?

We build the TP1 attack to see exactly which browser behaviours it relies on, then derive the defences: who must set them, and which can say "only these sites may frame me".



The portal frame sits *above* the game and is transparent; its Login button is aligned exactly over the game's button.

Alice's click goes to the top-most element: the *authenticated* portal's Login.

Alice is logged into the Student Portal. Eve's `game.html` embeds `portal.html` in an iframe positioned over the game with a higher `z-index` and `opacity: 0`; the game button moves randomly on each click, but at a chosen score it is placed exactly under the portal's Login. Alice clicks there; the click lands on the invisible Login; the portal, which received her session cookie when the iframe loaded, acts as Alice. The attack relies on three facts: a click goes to the top-most element and `opacity: 0` keeps the frame in hit-testing (`visibility: hidden` or `pointer-events: none` would not); the framed page received its cookies ("regardless of initiating site"; a Lax or Strict session cookie would *not* arrive, which mitigates); and nothing forbids framing unless the portal says so.

Definition 2.7.1 (Framing controls)**.** Both are HTTP response headers sent by the *framed* page and enforced by the browser. `X-Frame-Options: DENY` forbids all framing; `SAMEORIGIN` allows only same-origin framing; `ALLOW-FROM` is obsolete. CSP `frame-ancestors 'none'` (= DENY), `'self'` (= SAMEORIGIN), or a list of permitted origins (`frame-ancestors 'self' https://apps.um6p.ma`); it takes precedence over X-Frame-Options and is the only way to whitelist a specific third origin.

Technique: The TP's research questions

Protections: X-Frame-Options, CSP frame-ancestors (and JavaScript frame-busting, weak). Each controls who may embed the page. Same-origin only: `SAMEORIGIN` or `frame-ancestors 'self'`. A portal that must never be framed: `frame-ancestors 'none'` plus `X-Frame-Options: DENY` for old browsers. If a trusted university application must later embed it: `frame-ancestors 'self' https://trusted.um6p.ma`, which X-Frame-Options cannot express.

Hard questions, answered

Q. Setting anti-framing headers on the *game* page: useful?
A. No; the attacker controls the game page. The victim page must send them.

Q. Why is `opacity: 0` essential and `visibility: hidden` useless to the attacker?
A. Opacity keeps the frame in the hit-testing order, so it receives the click; visibility hidden removes it, so the click reaches the game.

Q. Which cookie attribute makes the clickjacked login click harmless, and why is it not the recommended defence?
A. `SameSite=Lax`/Strict on the session cookie: the cross-site iframe loads logged out. Still insufficient: actions needing no session remain hijackable and the page is still embeddable; frame-ancestors is the control.

Chapter takeaways

Summary: Stateful tracking

- Cookies are attached to every request to their domain, whoever initiated it; SameSite qualifies the second clause (Lax = top-level *and* safe method; None needs Secure); HttpOnly and Secure close script and network theft.
- Site (eTLD+1) governs attachment; origin governs script access; first party = the address bar's registrable domain.
- One tracker-domain cookie links all embedding sites (Theorem 2.4.1); publisher-domain analytics cookies cannot.
- Respawn from localStorage, Flash, ETag; clear all stores together; what survives everything is stateless.
- Syncing crosses domains inside a redirect URL; RTB broadcasts the bid request to every DSP.
- Clickjacking: transparent authenticated iframe; defence on the framed page: X-Frame-Options or CSP frame-ancestors.

Chapter 3

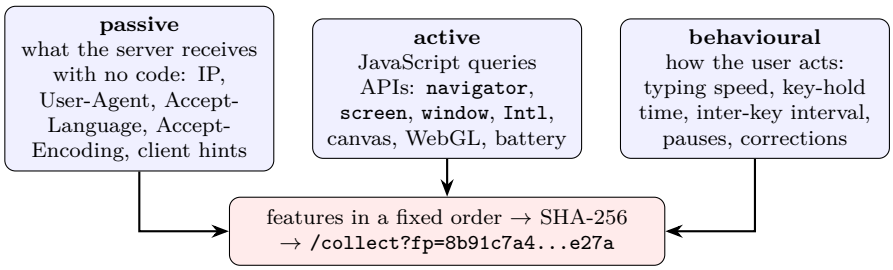
Stateless Tracking and Browser Defences

3.1 Recognition Without State

What are we trying to get?

Chapter 2 ended with a profile that survived the deletion of every store. We want the mechanism that explains it: recognising a browser from what it *reveals on every visit*, with no identifier stored; and the three ways that information is collected.

Definition 3.1.1 (Fingerprint; stateful vs stateless)**.** A **browser fingerprint** is the set of information about a device, from its hardware through its operating system to the browser and its configuration, that a web page can observe; **fingerprinting** is collecting it to recognise the device. **Stateful** tracking stores an identifier and reads it back later (cookies, storage, advertising IDs): "I recognise you because I stored something". **Stateless** tracking recomputes a recognisable pattern from properties exposed *now*: "I recognise you from what you reveal each time". Clearing state removes the stored identifier; it does not stop the pattern from being recomputed.



Reading the formula

The test for stateless is *where the identifier comes from*: an ETag is stateful (it sits in the cache) though it is not a cookie; a canvas hash is stateless though it looks like an ID. The three collection modes differ in what the tracker must do: passive costs nothing and needs no script; active needs JavaScript and reads APIs that the request never carried (screen size, fonts, time zone); behavioural measures the *user* rather than the device and is probabilistic (keyboard, fatigue, stress change it). Hashing the features in a fixed order

gives one short identifier: the same combination always maps to the same value, comparisons are cheap, and the user sees an unreadable string instead of the raw features, which is why fingerprinting is hard to notice.

The attack surface grew with the browser: Netscape Communicator 5.0 (1998, first open-source browser, ≈742k lines of code) to Chromium 120 (January 2024, ≈31M lines). Each new API (GPU, gyroscope, accelerometer, gamepad, USB, Bluetooth, AR/VR) is a feature and an attribute. A 1995 page learned "Netscape, French"; a 2024 page learns OS, screen, language, time zone, graphics card, audio and video capabilities.

Go deeper: the details an examiner asks about

- A fingerprint identifies a *browser instance*, never a named person (Quiz 1 Q11A false); exact matches produce false positives.
- Quiz 1 Q13: before any JavaScript runs, the server observes IP, User-Agent, Accept-Language, Accept-Encoding and client hints directly (A); screen resolution and fonts need active collection (B false); standard headers do distinguish browsers (C false); no cookie is needed first (D false).
- Disabling JavaScript stops active and behavioural collection; passive headers and stateful mechanisms remain.

Hard questions, answered

Q. Why is a hashed fingerprint not a permanent identifier?
A. Any changed feature (a browser update alters the User-Agent, a new font appears) changes the hash; following a browser over time needs *linking* (§3.4), not exact matching.

Q. Alice blocks cookies and uses private mode. Which of the three modes still work unchanged?
A. All three: none of them stores anything. Only tools that spoof or randomise attributes change the fingerprint, and those are themselves detectable (§3.4).

3.2 Entropy: How Much a Feature Identifies

What are we trying to get?

"This attribute is very identifying" needs a number. We want Shannon entropy built from the idea of surprisal, its additivity, and the scale (33 bits) on which fingerprints are judged.

Definition 3.2.1 (Surprisal; entropy)**.** If an attribute takes value x_i with probability $P(x_i)$ in the population, observing x_i yields $-\log_2 P(x_i)$ bits of information (its *surprisal*): a value shared by half the population yields 1 bit, one shared by a thousandth yields ≈ 10 bits. The expected surprisal is the

Shannon entropy

$$H(X) = -\sum_i P(x_i) \log_2 P(x_i).$$

Low entropy: many users share the value; high entropy: the value is rare and narrows the candidate set more.

Reading the formula

Each bit halves the candidate set. Distinguishing one device among 7×10^9 needs $\log_2(7 \times 10^9) \approx 33$ bits. Independent attributes add: $H(X, Y) = H(X) + H(Y)$; correlated ones (OS and installed fonts) add less than their sum. A uniform attribute over n values has $\log_2 n$ bits, the maximum; a constant attribute has 0. The *distribution* matters as much as the number of values: the skewed time-zone example below has three values and less than one bit.

Worked example: Quiz 1, Q15, and a skewed variant

100 fingerprints: 50 Windows, 25 Linux, 25 macOS, so $P = (0.5, 0.25, 0.25)$:

$$H = 0.5 \log_2 2 + 0.25 \log_2 4 + 0.25 \log_2 4 = 0.5 + 0.5 + 0.5 = 1.5 \text{ bits}$$

(answer C; the maximum for three values is $\log_2 3 \approx 1.585$). Time zone with 80% UTC+1, 10% UTC+0, 10% UTC+2: $H = 0.8 \times 0.322 + 2 \times 0.1 \times 3.322 = 0.258 + 0.664 = 0.92$ bits: three values too, but far less identifying.

Hard questions, answered

Q. An attribute has 16 equally likely values; another has 1000 values, one of which covers 99% of users. Which identifies more on average?
A. First: $\log_2 16 = 4$ bits. Second: $0.99 \log_2(1/0.99) + 0.01 \log_2(999/0.01) \approx 0.014 + 0.166 = 0.18$ bits: the rare values are informative but rarely observed; entropy is an expectation over the population.

Q. Why report entropy rather than "number of distinct values"?
A. Identification power is information observed, which depends on how values are distributed, and attribute entropies add only when attributes are independent; counts capture neither.

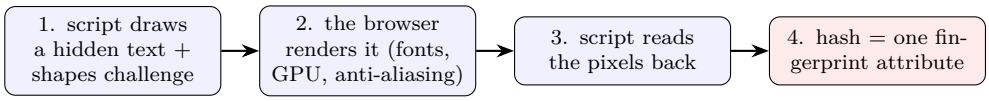
3.3 Attributes and Their Physics

What are we trying to get?

Each attribute on the syllabus has a reason it varies between devices. We want those reasons, because the examiner asks "how does canvas fingerprinting work" and "why can two identical laptops be told apart".

- **HTTP headers / User-Agent** (passive): grew from `NCSA_Mosaic/2.0 (Windows 3.1)` and `Mozilla/1.22 (compatible; MSIE 2.0; Windows 95)` into long strings naming engine, OS and versions; with `Accept`, `Accept-Encoding`, `Accept-Language`.

- **Screen, platform, time zone, cores, fonts, extensions** (active): e.g. `3840x2160x24`, `Win32`, UTC+8, 4 logical cores, the list of installed fonts.
- **Canvas** (Mowery & Shacham, W2SP 2012): JavaScript draws hidden text and shapes; the *instructions* are identical everywhere but the *rendered pixels* differ with browser, OS, fonts, GPU and graphics stack (anti-aliasing, hinting, sub-pixel rendering); the page reads the pixels back and hashes them. Quiz 1 Q14: render predefined content, read pixels, hash (B); it stores nothing in the canvas, retrieves no images, reads no serial number.
- **WebGL** vendor and renderer strings ("NVIDIA Corporation, GeForce GTX 4070 Ti").



- **GPU timing** (DrawnApart, Laor et al., NDSS 2022): two machines identical in model, OS, browser and GPU model still differ, because manufacturing variation makes some GPU execution units slightly faster. JavaScript measures timing traces of GPU work; a deep-learning model compresses each trace into an embedding; the same device yields similar embeddings despite noise, different devices yield different ones. On 2,550 devices and 370,392 fingerprints collected over 7 months in the wild, adding the GPU signal to FP-Stalker raised the median tracking duration in the 7-day revisit setting from 17.5 to 28 days (+10.5 days, ≈ 60%). The only *hardware-level* attribute on the syllabus; it improves *linkability*, not uniqueness.
- **Battery Status API** (first browser 2012; Olejnik, Acar, Castelluccia and Diaz, *The Leaking Battery*, 2015): charge level, charging state, estimated time remaining. A tracker on site A reads "73%, 14,237 s, not charging"; 20 seconds later on site B it reads "73%, 14,210 s, not charging". Few devices match that triple at that instant and the drift is predictable, so the two visits are linked *with no stored identifier*; minutes later the battery has drifted and the link expires. A temporary signal for short-term linking; the charge level could also be abused to raise prices.
- **IP address** (passive): WWW'20 study: 87% of devices kept the same IPv4 for more than 30 days (5,443 devices, 41,566 addresses), depending on country and ISP; the device–address graph was 93% unique; 20% of devices showed home–work–home cycles of 30 days or more. Shared behind NAT, so weak alone and strong in combination.
- **Mobile**: fingerprints are also highly unique (S&P'16), depending on vendor and model, through different attributes (User-Agent, emoji rendering, canvas).

Hard questions, answered

Q. Two laptops of the same model, OS, browser version and GPU model. Which attribute separates them and how?
A. DrawnApart: timing traces of GPU execution units reflect manufacturing variation invisible to every software attribute; a learned embedding of the trace is stable per device.

Q. Explain Battery-API linking and why it is not an identifier.

A. Level, time remaining and charging state at time t are rare in combination; at $t+20$ s the same device shows the same level and about 20 s less remaining, so a match is probable. Minutes later the triple has drifted and matches many devices: it links visits close in time only.

Q. Canvas needs JavaScript; the User-Agent does not. Why?
A. Canvas requires executing drawing code and reading pixels back (active); the User-Agent is a header the browser attaches to every request (passive).

3.4 Uniqueness, Linkability, Consistency

What are we trying to get?

Three research questions organise the field: can a fingerprint single you out, can it be followed as it changes, and do its attributes agree with each other. We want the numbers, the reason uniqueness collapsed between 2010 and 2018, and why that was not the end of tracking.

Uniqueness: how many are unique?
Panopticlick 470,161 fp, 94.2%
Beauty and the Beast (S&P'16) 118,934, 89.4%
Hiding in the Crowd (WWW'18) 1,816,776 desktop, 35.7%

Linkability: follow it as it evolves
FP-Stalker (S&P'18) 55 days avg
DrawnApart (NDSS'22) 17.5 → 28 days
Blink (SEAMS'15), FPRandom (ESSoS'17) break it

Consistency: do attributes agree?
FP-Scanner (USENIX Sec'18): spoofing tools are detectable and some leave per-user traces

Uniqueness fell from 94% to 36% between the early study and *Hiding in the Crowd* for three reasons: a far larger and more diverse sample (nearly two million desktop fingerprints on a popular site); the disappearance of high-entropy attributes (Flash, Java, enumerable plugin lists); and homogeneous populations on identical OS and browser versions. **Linkability** is about *evolution*: fingerprints change with updates, yet FP-Stalker follows a browser for 55 days on average with the right linking algorithm, and the GPU signal extends it; Blink breaks it by generating a random disposable environment (OS, browser, fonts) per session, FPRandom by adding controlled noise to canvas (adopted by Brave). **Consistency** cuts both ways: a tool that spoofs the User-Agent to say Windows while fonts, platform and WebGL still say Linux creates a detectable contradiction; FP-Scanner shows most such tools are detectable and some add a unique per-user trace, so a badly designed defence becomes a new attribute. Other works on the map: Fingerprinting in Style (USENIX Sec'21), Taming the Shape Shifter and Web Runner 2049 (DIMVA'20), Morellian analysis (DIMVA'19), FP-Redemption (DIMVA'21). `amiunique.org` (November 2014) shows your own fingerprint and, through an extension, its evolution.

Interpretation & discussion

36% unique is not the end of tracking. Non-unique fingerprints lie in *small* groups; add an IP address, a login or a short-term battery signal and the group collapses to one device; and following a changing fingerprint for weeks needs linkability, not global uniqueness. The same data has a defensive use: "does this login look familiar?" compares the current browser with past logins of a

known account; the fingerprint is a *risk signal*, never proof of identity (false positives, drift), so a mismatch should trigger step-up authentication rather than a denial or a silent acceptance.

Go deeper: the details an examiner asks about

- Quiz 1 Q11: Test 4 (all stores cleared; fingerprint 88 and profile 3 persist) is *consistent with* stateless relinking (B) and other browsers may share 88 (D); it proves neither a named identity (A) nor a failed clearing (C).
- Naive randomisation attacks linkability but creates inconsistency; a tool used by few people is itself rare. Brave's site-dependent noise (§3.6) is the designed answer.

Hard questions, answered

Q. A defence randomises canvas on every page load. Which property does it attack, and what can go wrong?
A. Linkability (and the canvas attribute's uniqueness). Risks: FP-Scanner-style coherence checks detect the tool; the tool's rarity becomes an identifying attribute; sites needing stable canvas output break.

Q. Why is 35.7% unique still a privacy problem?
A. Small anonymity groups collapse under one extra signal, and FP-Stalker-style linking follows a browser for weeks without uniqueness.

3.5 Blocking: Filter Lists and Behaviour

What are we trying to get?

Two philosophies of blocking: know the trackers in advance (lists) or recognise tracking behaviour as it happens. We want the list syntax the course tests, the four limitations of lists, and how Privacy Badger decides.

Definition 3.5.1 (Filter list; EasyList syntax)**.** A **filter list** is a maintained set of rules describing network requests or page elements that a blocker should block, allow, redirect or hide: the Disconnect list (Firefox, Brave), EasyList (ads) and EasyPrivacy (tracking) (AdBlock Plus, uBlock Origin). A network rule `||tracker.com^` blocks every request whose URL matches: `||` anchors at the start of a domain (so `sub.tracker.com` matches but `nottracker.com` does not), `^` marks the end of the domain or a separator such as `/`. A rule beginning with `@@` is an *exception* that allows something a broad rule would break.

Lists are *knowledge-based*: "we already know this domain tracks; if it matches, block". Their limitations: *coverage* (a tracker with no matching rule loads), *evasion* (trackers rotate domains, change URLs, or hide as first-party through CNAME), *breakage* (aggressive rules stop logins, video, payment, embeds), *maintenance* (continuous measurement, reports and updates). **Behaviour-based** protection asks what a third party *does*: does it appear across many websites, try to recognise the same browser, store identifying information? Privacy Badger (EFF)

watches for identifying cookies, local-storage "supercookies" and fingerprinting, and restricts or blocks a third party observed tracking across several unrelated sites; the threshold balances false positives on ubiquitous CDNs against coverage. FP-detect turns the tables: it detects which lists and extensions are active, so the blocker becomes an attribute.

Worked example: EasyList exercise

With `||amazon-adsystem.com^` in the list, both `/ad.js` and `/widgets/q?item=10` on that domain match the domain rule and are blocked *unless* an `@@` exception (for example on `/widgets/`) exists; exceptions exist precisely to prevent breakage, so one checks the list instead of guessing. "Block all of `ads.example.net` except `/consent/`": `||ads.example.net^` plus the exception `@@||ads.example.net/consent/`.

Hard questions, answered

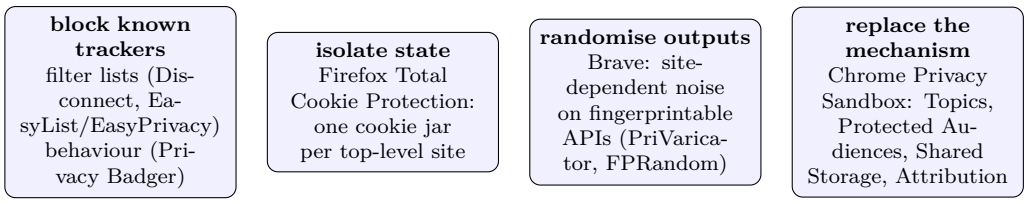
Q. A tracker is served from `static.publisher.com`. Which limitation is this and which protection can still catch it?
A. Evasion via first-party or CNAME hosting; URL lists miss it. DNS-level inspection (Firefox resolves the CNAME chain) or behaviour-based detection if it tracks across sites (it usually appears on one, so even Privacy Badger may miss it).

Q. Why can installing a blocker make you *more* fingerprintable?
A. Blockers change observable behaviour (which requests fail, which elements vanish); FP-detect infers the active lists and extensions, adding a rare attribute.

3.6 Browsers Against Tracking

What are we trying to get?

The browser vendors' responses differ in philosophy and in what they leave open. We want each mechanism, the timeline of the third-party-cookie saga, and the two evasion techniques (Topics misuse, CNAME cloaking) that show the limits.



- **Third-party cookie deprecation.** Google announced the end of third-party cookies in Chrome (January 2020): planned for Q2 2022, delayed to Q4 2023, then to Q4 2024, "the biggest change in ad tech since its inception"; in 2024 Chrome moved to a *user-choice* approach; in 2025–2026 Google confirmed no universal phase-out. The lesson: browser privacy is shaped by compatibility, advertising economics, standards and deployment, not only by technique. Advertisers meanwhile invested in their own user IDs, device data and first-party data (precise, easy to obtain, compatible with legislation),

and every platform opened a Conversions API (Meta, Amazon, Microsoft Offline Conversions, Google Enhanced Conversions, TikTok Events, X).

- **Privacy Sandbox** (Chromium): Topics API (interest-based targeting), Protected Audiences (retargeting), Shared Storage (frequency capping), Attribution Reporting. *Topics*: each week the browser derives 5 topics from the browsing history and adds one random topic, from a taxonomy of 469; the random topic gives deniability. Apple and Mozilla objected; PETS'24 showed that advertisers and publishers can abuse Topics to assign unique identifiers (the sequence of weekly topic sets observed by several sites is specific enough to link a browser) and that the design is "at odds with the natural diversity of user behaviours".
- **Firefox Total Cookie Protection**: each top-level site gets its own cookie jar; a tracker embedded on site A and site B receives two different cookies, so Theorem 2.4.1 fails and the tracker degrades to per-site analytics. It targets stateful cross-site tracking only: fingerprinting needs separate defences, CNAME-cloaked first-party cookies are untouched, and a sync between two trackers on the *same* site still works.
- **Brave**: add subtle, *site-dependent* randomness to selected fingerprintable outputs so repeated measurements are less useful as a stable identifier: the same site sees stable values (nothing breaks) while different sites see different values (no linking). Contrast with inconsistent spoofing (§3.4).
- **CNAME cloaking**: DNS maps `f0Ab1ze.mywebsite.com CNAME bad.tracker.com`, `bad.tracker.com A 119.183.2.123`. The browser sees a first-party hostname: cookies are first-party, URL filter lists miss it, and only resolving the CNAME chain (Firefox exposes DNS-level APIs to blockers) reveals the tracker. A threat to transparency because the mapping lives in DNS and server configuration; regulators such as CNIL must audit companies to see it.

Go deeper: the details an examiner asks about

- Dates: 2020 announcement; planned Q2 2022, Q4 2023, Q4 2024; 2024 user choice; 2025–26 no phase-out.
- Quiz 1 Q6 under Total Cookie Protection: (A) and (D) become false, two jars, two `tid` values.
- A browser needs all four columns of the figure: lists for known trackers, isolation for stateful reuse, randomisation for fingerprints, and an honest replacement for the advertising use case.

Hard questions, answered

Q. Why can Topics approximate an identifier although each week exposes only 5 of 469 topics plus a random one?
A. Several sites observe the same browser's weekly sets over many weeks; the *sequence* of sets is highly specific to one history, so the sites link the browser by matching sequences; one random topic per week is too little noise across weeks.

Q. Why is CNAME cloaking "a big threat for transparency"?
A. Nothing in the page reveals the tracker: the hostname is first-party and

the alias lives in DNS; users and URL-based blockers see a first-party request; only DNS inspection or an audit of the company exposes it.

Q. Compare Firefox partitioning and Brave randomisation: which threat does each address, and what does each miss?
A. Firefox: stateful cross-site tracking through cookie reuse; misses fingerprinting, CNAME-cloaked first-party cookies, same-site syncing. Brave: fingerprint stability; misses stateful tracking unless combined with blocking or partitioning.

Chapter takeaways

Summary: Stateless tracking and defences

- Stateless = recompute from exposed properties; passive, active, behavioural; a hash makes one unreadable identifier of a *browser*, not a person.
- $H = -\sum P \log_2 P$; independent attributes add; 33 bits single out a device; skew lowers entropy.
- Attributes: headers; screen, fonts, time zone; canvas rendering; WebGL; GPU timing (hardware); battery (short-term); IP (stable for weeks).
- Uniqueness fell to 36% on desktop; linkability (55 days; 28 with GPU) is the real threat; inconsistent spoofing is detectable.
- Lists (`||domain^`, `@@`) are knowledge-based and evadable (CNAME); Badger is behaviour-based; Firefox partitions, Brave randomises per site; Topics is a quasi-identifier; Chrome kept third-party cookies.

Chapter 4

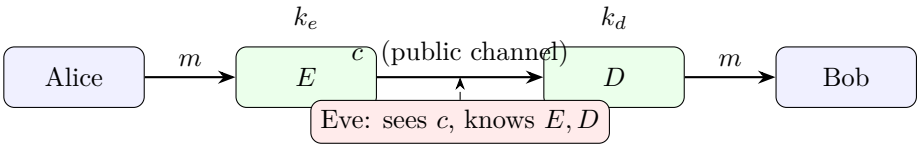
Symmetric Cryptography

4.1 Cryptosystems, Kerckhoffs, Attack Models

What are we trying to get?

We want the objects (message, key, cipher), the one design principle that governs all modern cryptography (Kerckhoffs), and the attack models in which security is stated; without a model, "this cipher is secure" is the under-defined circle question of Chapter 1.

Definition 4.1.1 (Cryptosystem)**.** *Cryptology* = *cryptography* (secret writing, hiding meaning) + *cryptanalysis* (breaking it). A cryptosystem is (M, C, K, E, D) with encryption $c = E(m, k_e)$ and decryption $m = D(c, k_d)$; *symmetric* if $k_e = k_d$ (everyone who knows k knows the whole secret), *asymmetric* if $k_e \neq k_d$ and k_d is infeasible to derive from k_e. The **key space** K is the set of keys; **brute force** tries them all. Cryptography provides *confidentiality* (against eavesdroppers), *integrity* (content unaltered) and *authentication* (origin as claimed); a cipher by itself gives only the first.



Key insight

Kerckhoffs's principle: the system must remain secure when everything except the key is public. Eve in the picture knows E and D. Consequences: algorithms are published and reviewed (DES and AES came out of open competitions); a compromised key is replaced without redesigning anything; a "secret algorithm" is a red flag because algorithms leak through reverse engineering and cannot be rotated. Historically: simple algorithms with long keys; today: sophisticated public algorithms with short keys.

Definition 4.1.2 (Attack models)**.** What the cryptanalyst holds besides the algorithm and the target ciphertext: **ciphertext only** (COA); **known plaintext** (KPA: some plaintext–ciphertext pairs under the same key); **chosen plaintext** (CPA: plaintexts of the attacker's choice get encrypted); **chosen**

ciphertext (CCA: ciphertexts of the attacker's choice get decrypted); **chosen text** (both). Power grows COA < KPA < CPA, CCA, and security under a stronger model implies the weaker ones. Four ways to attack: *analytical* (cleverness: a weakness in the algorithm or its use), *brute force* (money: parallel key search; *computationally secure* means the cost of cracking exceeds the value of the information), *social engineering* (the person), *implementation attacks* (software, hardware, side channels, usage).

Go deeper: the details an examiner asks about

- Modern ciphers must resist *chosen*-plaintext attacks because attackers routinely control inputs (a server encrypting their data); CPA security implies KPA and COA security.
- "Computationally secure" is economic (cost > value); "unconditionally secure" (the one-time pad) is information-theoretic.

Hard questions, answered

Q. A realistic setting for each attack model?
A. COA: intercepted radio traffic. KPA: encrypted files with known headers (PDF magic bytes). CPA: a web application that encrypts a cookie value the attacker supplies. CCA: a server that decrypts submitted tokens and reveals whether the padding was valid.

Q. Classify: power-trace analysis of a smart card; bribing the operator; trying all 2^{56} DES keys; exploiting ECB block independence.
A. Implementation attack; social engineering; brute force; analytical attack.

4.2 Classical Ciphers and Their Cryptanalysis

What are we trying to get?

Three historical ciphers teach three lessons: a tiny key space falls to brute force (Caesar); a huge key space is not enough when statistics survive (substitution); a repeating key reduces to the previous case (Vigenère). We prove the second and give the algorithm for the third.

Definition 4.2.1 (Caesar, substitution, Vigenère)**. Caesar**: $c = (p + k) \bmod 26$ on letters 0..25; key space 26 (`Security is very interesting` with $k = 3 \to$ `VHFXULWB LU YHRB LQWHUHVWLQJ`). **Simple substitution**: a fixed permutation σ of the alphabet, $c = \sigma(p)$; key space $26! \approx 4.03 \times 10^{26}$. **Vigenère**: a keyword $k_0 \dots k_{L-1}$ repeats and letter i is shifted by $k_{i \bmod L}$ (`send reinforcements` under `somesecret` $\to$ `kszhjikejhjqqqwrvj`).

Theorem 4.2.2 (Substitution preserves the frequency histogram)**.** *Under a monoalphabetic substitution, the ciphertext letter counts are a relabelling of the plaintext letter counts. Hence the key is recovered letter by letter from language statistics, in time independent of the key-space size.*

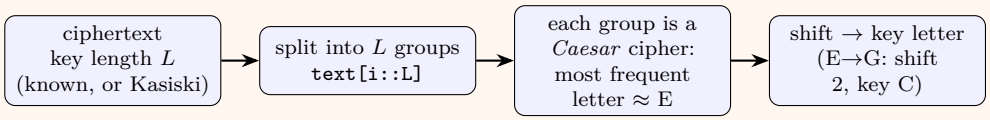
Proof. For each plaintext letter a, $\#\{i : c_i = \sigma(a)\} = \#\{i : p_i = a\}$ because σ

is a bijection applied position-wise. The multiset of counts is unchanged; only the labels move. English counts are far from uniform (E 12.7%, T 9.1, A 8.2, O 7.5, I 7.0, N 6.7, S 6.3, H 6.1, R 6.0, D 4.3, L 4.0, C 2.8; J, X, Q, Z rarest), so sorting the ciphertext counts largely determines σ; short words, doubled letters and repeated patterns resolve the remaining ambiguity. □

Key insight

A large key space is necessary, not sufficient. Brute force is defeated by 26!; analysis is not, because the cipher leaves the *structure* of the plaintext intact. Every modern design goal (confusion, diffusion, §4.4) exists to destroy exactly this structure.

Technique: Breaking Vigenère



Every letter in group i was shifted by the same k_i, so Theorem 4.2.2 applies within the group with the simplest possible σ. Unknown L: repeated trigrams in the ciphertext occur at distances that are multiples of L (Kasiski), or compare the index of coincidence for candidate lengths. The attack needs the key to *repeat*: a random key as long as the message, never reused, leaves one letter per group and no statistics (§4.3).

Worked example: The TP escape room

Caesar $k = 4$: `CRYPTO IS FUN UNTIL THE PROFESSOR SAYS QUIZ` → `GVCTXS MW JYR YRXMP XLI TVSJIWWSV WECW UYMD`. Decrypt with $k = 8$: `GWCL JMBBMZ VWB JM CAQVO BWWTA BW AWTDM BPQA` → `YOUD BETTER NOT BE USING TOOLS TO SOLVE THIS`. Unknown key, try all 26: `ESP VPJ TD FYVYZHY ECJ MCFEP QZCNP` is shift 11, `THE KEY IS UNKNOWN TRY BRUTE FORCE`. Substitution table (A→Q, B→M, C→J, D→Z, E→T, ...): `KEEP THE SECRET SAFE` → `LTTN EKT YTJRTE YQGT`; `ERVYE DXVR SXFPJ` → `TRUST YOUR LOGIC`. The slide's `GSRH RH Z HVXIVG` is the reversed alphabet (Atbash): `THIS IS A SECRET`.

Go deeper: the details an examiner asks about

- Enigma gave the German navy an early advantage in WWII; broken with Turing's *bombe*. The lesson the course draws: with computers, substitution and transposition alone are obsolete; modern ciphers combine substitution with repeated transposition so that ciphertext looks random (DES, AES).
- The TP hint order for its specific text, E > T > R > S/O > A > H > N > D > I > G > L, shows that frequency analysis gives a *hypothesis*; context finishes the job.

Hard questions, answered

Q. Why is a substitution cipher with 26! keys broken "faster" than Caesar is brute-forced?
A. Both are trivial, but for different reasons: Caesar by 26 trials, substitution by analysis of surviving statistics. The point is that 26! bought no security because the attack never enumerates keys.

Q. Vigenère with a 1000-letter random key on a 1000-letter message, used once. Breakable by the group method?
A. No: $L = 1000$, one letter per group, no statistics; that is the one-time pad. The attack needs repetition.

4.3 XOR and the One-Time Pad

XOR is "different $\to$ 1, same $\to$ 0", and $(x \oplus k) \oplus k = x$, so one operation both encrypts and decrypts. `Hello` $\oplus$ the repeating key `key` gives `00100011 00000000 00010101 00000111 00001010`; XOR with the key again returns `Hello`.

Theorem 4.3.1 (One-time pad)**.** *If the key is uniformly random, as long as the message, and used once, then for every ciphertext c and every candidate plaintext m of that length there is exactly one key with $m \oplus k = c$; a ciphertext-only attacker learns nothing about m.*

Proof. $k = m \oplus c$ is the unique key; since all keys are equally likely, all plaintexts of that length are equally consistent with c. □

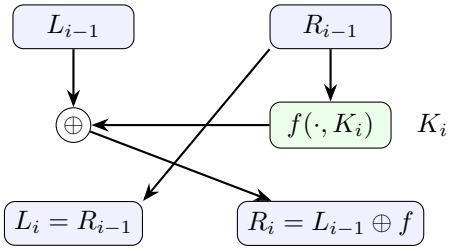
With a *repeating* key the scheme is Vigenère on bytes and fails in every model: KPA gives $m \oplus c = k$ directly; CPA with an all-zero plaintext returns k as the ciphertext; CCA with an all-zero ciphertext returns k as the "plaintext"; COA notices that the second ciphertext byte is `00000000`, hence the second key byte equals the second plaintext byte, and proceeds from language structure. Reuse is the fatal case: $c_1 \oplus c_2 = m_1 \oplus m_2$ and two structured texts can be pulled apart (crib dragging).

Hard questions, answered

Q. Why is the one-time pad rarely used although it is perfectly secure?
A. The key must be as long as all traffic, truly random, delivered secretly, and never reused; the key-distribution problem is as hard as the message-distribution problem. Block ciphers use a short key to *generate* a long pseudorandom stream (CTR mode) instead.

4.4 Confusion, Diffusion, Feistel

Definition 4.4.1 (Confusion and diffusion (Shannon))**. Confusion**: make the relation between key, plaintext and ciphertext complex (nonlinear); Caesar's $c = (p+k) \bmod 26$ has none. **Diffusion**: spread each input bit's influence over many output bits so that local statistics disappear. Block ciphers obtain both by iterating *rounds* that substitute (confusion) and permute or mix (diffusion).



A **Feistel round**: only the right half enters f; the left half is XOR-ed with the result; the halves swap. Decryption runs the same round with the same K_i because XOR cancels.

Theorem 4.4.2 (Feistel networks invert for free)**.** *For any function f, the round $(L_{i-1}, R_{i-1}) \mapsto (L_i, R_i)$ with $L_i = R_{i-1}$ and $R_i = L_{i-1} \oplus f(R_{i-1}, K_i)$ is inverted by $R_{i-1} = L_i$, $L_{i-1} = R_i \oplus f(L_i, K_i)$. An n-round Feistel cipher is therefore decrypted by the encryption algorithm with the subkeys in reverse order, and f need not be invertible.*

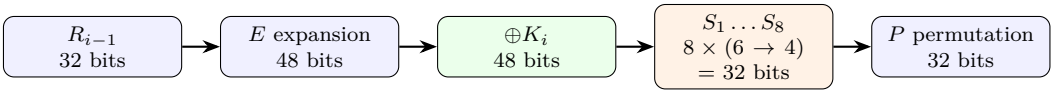
Proof. L_i is a copy of R_{i-1}, so $f(R_{i-1}, K_i) = f(L_i, K_i)$ is recomputable from the output; XOR-ing it into R_i cancels it since $x \oplus y \oplus y = x$. Apply from round n down to 1. □

4.5 DES

What are we trying to get?

DES is the worked example of a Feistel cipher: we want its parameters, the four steps of its round function with one round computed by hand, the key schedule and why reversing it decrypts, and why it fell.

Definition 4.5.1 (DES)**.** Block 64 bits, key 56 bits (the 64-bit input key carries 8 parity bits at positions 8, 16, …, 64, dropped by permuted choice PC-1), 16 Feistel rounds with distinct 48-bit subkeys, an initial permutation IP and its inverse IP^{-1} at the ends. The round function $f(R, K_i)$, 32 bits in and out: (1) **expansion** E, 32 → 48 bits, by duplicating the two boundary bits of each 4-bit group; (2) XOR with K_i; (3) eight **S-boxes** $S_1..S_8$, each 6 bits → 4: the *row* is the outer bits b_1b_6, the *column* the inner bits $b_2b_3b_4b_5$; the only nonlinear step and "crucial for DES security"; (4) **permutation** P, a fixed rearrangement sending each S-box's output bits to different S-boxes in the next round (diffusion). IP, E and P are position tables: they move bits and never change their values.



K_i from the key schedule

Worked example: One round: the slide's numbers and the toy round

Slide: input `0x0123456789ABCDEF`; IP's first row (58, 50, 42, 34, 26, 18, 10, 2) selects input bits that form the first output byte `11001100`. After IP, $R_0 =$ `0xF0AAF0AA`; $E(R_0) =$ `011110 100001 010101 010101 011110 100001 010101 010101`; after $\oplus K_1$: `0x6117BA866527`; the first 6-bit group `011000` has row 00 = 0 and column 1100 = 12, and $S_1[0, 12] = 5 =$ `0101`; all eight boxes give `0x5C82B597`; P yields `0x234AA9BB`. Toy round ($L_0 =$ `1010`, $R_0 =$ `1001`, $E' = [4, 1, 2, 3, 4, 1]$, $K_1 =$ `101011`, $P' = [2, 4, 3, 1]$): $E'(R_0) =$ `110011`; $\oplus K_1 =$

011000; $S_1 \to$ 0101; $P' \to$ 1100; $L_1 = R_0 =$ 1001; $R_1 = L_0 \oplus$ 1100 = 0110.

Definition 4.5.2 (Key schedule; decryption)**.** PC-1 keeps 56 bits and splits them into 28-bit halves C_0, D_0. In round i each half is rotated left by 1 bit in rounds 1, 2, 9, 16 and by 2 bits in all other rounds; PC-2 then selects a permuted 48-bit subset as K_i. The total rotation is $4 \cdot 1 + 12 \cdot 2 = 28$, so $C_{16} = C_0$ and $D_{16} = D_0$. Decryption uses the same 16 keys in order $K_{16} \dots K_1$ (Theorem 4.4.2); equivalently, run the schedule with no rotation in round 1, one bit to the *right* in rounds 2, 9, 16 and two bits right elsewhere. No new keys are generated.

Worked example: Toy key schedule

Key 10101101; PC-1 $= [1,2,3,5,6,7]$ keeps 101110; $C_0 =$ 101, $D_0 =$ 110. Round 1, shift 1: $C_1 =$ 011, $D_1 =$ 101; PC-2 $= [1,2,4,5]$ on 011101 gives $K_1 =$ 0110. Round 2 continues from C_1, D_1 (never restart from the master key): $C_2 =$ 110, $D_2 =$ 011; 110011 $\to K_2 =$ 1101.

Interpretation & discussion

Strength: $2^{56} \approx 7.2 \times 10^{16}$ keys. At 10^9 keys per second one machine needs 7.2×10^7 s ≈ 2.3 years; the EFF's $250,000 machine (1998) cracked DES in under two days. DES died of key size, not of cryptanalysis. **3DES** applies DES three times in encrypt–decrypt–encrypt order (so $K_1 = K_2 = K_3$ degrades to DES for compatibility) with two keys (112 bits effective) or three (168). *Double* DES is pointless: *meet in the middle* encrypts a known plaintext under all 2^{56} values of K_1, decrypts its ciphertext under all 2^{56} values of K_2, and matches the tables, for $\approx 2^{57}$ work instead of 2^{112}. 3DES survives in legacy finance (ATMs); AES replaced it and runs 5× faster in CPU hardware.

Go deeper: the details an examiner asks about

- Why DES decrypts with reversed keys and AES does not: Theorem 4.4.2 needs the Feistel structure; AES acts on the whole block and needs explicit inverse operations.
- S-box indexing trap: "outside bits = row, inside bits = column". Row 2 of S_1: 4 1 14 8 13 6 2 11 15 12 9 7 3 10 5 0.
- Timeline: DES 1975 (56/64); 3DES 1995 (112/168, 64); Twofish, Serpent, Rijndael 1998 (128/192/256, 128); AES standard 2001 (FIPS-197).

Hard questions, answered

Q. S_1 on 100110?
A. Row $b_1 b_6 =$ 10 $= 2$, column 0011 $= 3$: $S_1[2,3] = 8 =$ 1000.

Q. Why 56 key bits from a 64-bit input, and why is 2DES only ≈ 57 bits strong?
A. Eight parity bits are discarded by PC-1. Meet in the middle replaces one 2^{112} search by two 2^{56} tables.

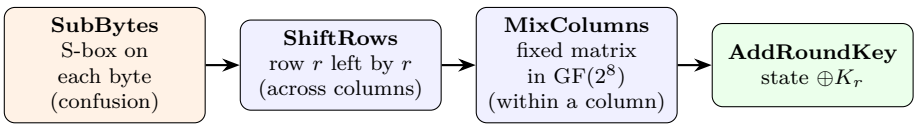
4.6 AES

What are we trying to get?

AES is the substitution–permutation design: we want the byte arithmetic it lives in, its four round operations with the diffusion they produce, one MixColumns byte by hand, the key expansion, and the decryption order; the TP's "AES without diffusion" shows why each piece is needed.

Definition 4.6.1 (GF(2^8) arithmetic)**.** Bytes are polynomials of degree < 8 with coefficients in $\{0, 1\}$. Addition is XOR. Multiplication is polynomial multiplication reduced modulo $m(x) = x^8+x^4+x^3+x+1$ (`0x11B`). Multiplying by x (`02`, `xtime`): shift left one bit, and if the bit shifted out was 1, XOR with `1B` (because $x^8 \equiv x^4 + x^3 + x + 1$). $03 \cdot b = (02 \cdot b) \oplus b$. Every nonzero byte has an inverse, which is what makes SubBytes and MixColumns invertible.

Definition 4.6.2 (AES-128)**.** Rijndael (Rijmen and Daemen; FIPS-197, 2001). Block 128 bits held as a 4×4 byte *state* filled column by column (b_{i+4j} at row i, column j); keys of 128/192/256 bits with 10/12/14 rounds and $N_r + 1$ round keys; *not* Feistel. Encryption: AddRoundKey(K_0); rounds $1..N_r - 1$ of **SubBytes** (each byte through the S-box; confusion), **ShiftRows** (row r rotated left by r; moves bytes *between* columns), **MixColumns** (each column multiplied by the fixed matrix in GF(2^8); mixes *within* a column), **AddRoundKey** (XOR with K_r); a final round without MixColumns. ShiftRows and MixColumns together give diffusion: after two rounds every byte depends on every input byte. Decryption: AddRoundKey(K_{10}); for rounds 9..1: InvShiftRows (right rotations), InvSubBytes (inverse S-box), AddRoundKey, InvMixColumns (matrix `0E 0B 0D 09`); finally InvShiftRows, InvSubBytes, AddRoundKey(K_0).



one round; N_r = 10/12/14 rounds; AddRoundKey(K_0) comes first; the last round skips MixColumns

Worked example: MixColumns, first byte of column `D4 BF 5D 30`

$b_0 = (02 \cdot \text{D4}) \oplus (03 \cdot \text{BF}) \oplus 5\text{D} \oplus 30$. $02 \cdot$ D4: `11010100` $\rightarrow$ `10101000` = `A8`, a 1 was shifted out, so $\oplus$`1B` = `B3`. $02 \cdot$ BF: `10111111` $\rightarrow$ `01111110` = `7E`, carry, $\oplus$`1B` = `65`; $03 \cdot$ BF = 65 $\oplus$ BF = DA. Then B3 $\oplus$ DA = 69, 69 $\oplus$ 5D = 34, 34 $\oplus$ 30 = 04. The full column becomes `04 66 81 E5`; AddRoundKey with `A0` gives `A4`. Other slide values: SubBytes `19`$\rightarrow$`D4`, `00`$\rightarrow$`63`, `03`$\rightarrow$`7B`, `71`$\rightarrow$`A3`; ShiftRows on the index state: row 1 `11 12 13 10`, row 2 `22 23 20 21`, row 3 `33 30 31 32`; AddRoundKey with the all-`0F` key XORs each byte with `0F` (`23`$\rightarrow$`2C`).

Definition 4.6.3 (Key expansion (AES-128))**.** 44 four-byte words; $w_0..w_3$ are the key. For $i \geq 4$: $t = w_{i-1}$; if $i \equiv 0 \pmod 4$, $t \leftarrow$ RotWord (rotate one byte), SubWord (S-box each byte), XOR `Rcon`$[i/4]$ into the first byte; then

$w_i = w_{i-4} \oplus t$. Round key r is $w_{4r}..w_{4r+3}$. `Rcon` first bytes: `01 02 04 08 10 20 40 80 1B 36`, the successive powers of `02` in the field (hence `1B` after `80`).

From the TP: AES without diffusion

Remove ShiftRows and MixColumns from a two-round cipher on one byte: $c = S(S(p \oplus k_0) \oplus k_1) \oplus k_2$. From the pair $(00, \text{A4})$, $k_2 = \text{A4} \oplus S(S(k_0) \oplus k_1)$ is a function of (k_0, k_1); enumerate the 2^{16} pairs, derive k_2, keep those consistent with $(01, \text{F8})$ and $(02, \text{3F})$. Each byte position is attacked *independently* with three known plaintexts; with diffusion the 16-byte state must be attacked as a whole (2^{128}). Confusion without diffusion is attackable byte by byte.

Go deeper: the details an examiner asks about

- The last round omits MixColumns because a trailing public linear map adds no security and would break the symmetry between encryption and decryption.
- SubBytes is the only nonlinear step; without it AES is a linear map, solvable by linear algebra from a few plaintext–ciphertext pairs.
- AES-128 vs AES-256: key length, rounds (10 vs 14), round keys (11 vs 15), expansion details; block size and the four operations are unchanged.

Hard questions, answered

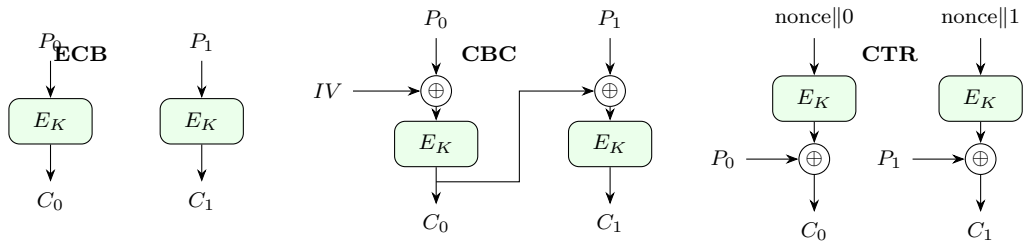
Q. Compute $02 \cdot 57$ and $03 \cdot 57$.
A. `57` = `01010111` → `10101110` = `AE`, no carry: $02 \cdot 57 = \text{AE}$; $03 \cdot 57 = \text{AE} \oplus 57 = \text{F9}$.

Q. Which step gives confusion, which give diffusion, and what breaks without each?
A. SubBytes: confusion (remove it: linear, trivial). ShiftRows + MixColumns: diffusion (remove them: the TP's byte-wise 2^{16} attack). AddRoundKey injects the key (remove it: no secret at all).

4.7 Modes of Operation

What are we trying to get?

A block cipher encrypts one block; a mode says how to encrypt a message of many blocks. We want the three modes of the course with their equations, their parallelism and error behaviour, and the one visual fact every student must be able to explain: why ECB shows the penguin.



Definition 4.7.1 (ECB, CBC, CTR)**.** **ECB**: $C_i = E_K(P_i)$; parallel both ways; deterministic per block, so equal plaintext blocks give equal ciphertext blocks and the *structure* of the data leaks. **CBC**: $C_i = E_K(P_i \oplus C_{i-1})$ with $C_0 = IV$, and $P_i = D_K(C_i) \oplus C_{i-1}$; encryption is sequential (needs C_{i-1}), decryption parallel with random access (needs only C_i, C_{i-1}); the **IV** is random and *not secret* (sent in clear); it prevents a repeated first block from producing a repeated ciphertext. **CTR**: $C_i = P_i \oplus E_K(\text{nonce}\|i)$, a stream cipher built from a block cipher, fully parallel, random access; the nonce must never repeat under one key (two-time pad). Others: CFB, OFB, and CCM = CTR for confidentiality plus CBC-MAC for integrity (authenticated encryption).

Theorem 4.7.2 (Error propagation in CBC)**.** *A flipped bit in ciphertext block C_i garbles plaintext block P_i entirely and flips exactly that bit in P_{i+1}; all later blocks decrypt correctly.*

Proof. $P_i = D_K(C_i) \oplus C_{i-1}$ pushes the corrupted block through D_K, whose output is pseudorandom. $P_{i+1} = D_K(C_{i+1}) \oplus C_i$ XORs the corrupted C_i directly, bit for bit. P_{i+2} depends only on C_{i+1}, C_{i+2}. □

From the TP: OpenSSL mission 2: ECB on an image

OpenSSL's salted format starts with the 8-byte signature `Salted__` followed by an 8-byte *salt* (public; the key is derived from password and salt with PBKDF2), so the first 16 bytes are not ciphertext. The image is $432 \times 475 = 205{,}200$ bytes, exactly 12,825 blocks, so PKCS#7 appends one full padding block: ciphertext 205,216 bytes, file 205,232. Counting equal 16-byte blocks (`collections.Counter`) and giving equal blocks the same grey shade reproduces the picture's outline: a *pattern, not the secret*, since the pixel values stay hidden. Decrypt with `openssl enc -d -aes-256-ecb -pbkdf2 -p -in classified.enc -out recovered.raw`; re-encrypting with `-aes-256-cbc` leaves essentially no repeated blocks because each block is XOR-ed with the previous ciphertext first.

Go deeper: the details an examiner asks about

- A random IV makes encryption *probabilistic*: the same message twice gives different ciphertexts; a fixed IV reintroduces an ECB-like leak for the first block.
- CBC without a MAC allows bit-flipping attacks on P_{i+1} by Theorem 4.7.2: confidentiality is not integrity (Chapter 5).

Hard questions, answered

Q. Why is a clear-text IV not a weakness, and what would be?
A. The IV only needs to be unpredictable or unique so that equal first blocks differ; knowing C_0 does not help invert E_K. Reusing an IV, or using a predictable one under CPA, is the weakness.

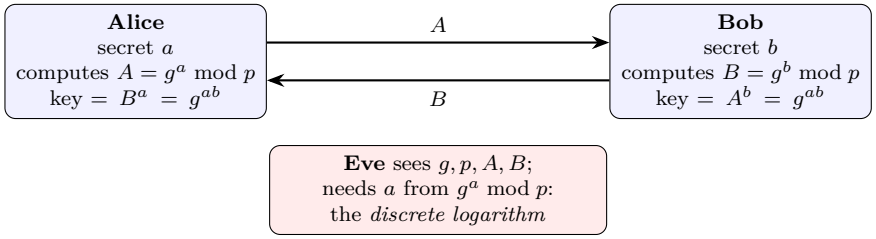
Q. Rank ECB, CBC, CTR on encryption parallelism / decryption parallelism / random access / pattern hiding.
A. ECB yes/yes/yes/no; CBC no/yes/yes/yes; CTR yes/yes/yes/yes (with a

unique nonce).

Q. Strong cipher, weak system: AES-256-ECB on a column of 16-byte customer IDs.
A. Equal IDs give equal ciphertexts; frequency and equality analysis works without touching AES. Use CBC or CTR with fresh IVs, or authenticated encryption.

4.8 Key Exchange: Diffie–Hellman

Symmetric ciphers need the key delivered over a secure channel. Ralph Merkle proposed public-key ideas as a Stanford student in 1974 (shot down by his professor) and in 1975 (rejected by CACM as "extremely dangerous to transmit key information in the clear"); Diffie and Hellman published in 1976.



Public g and p; both compute the same $g^{ab} \bmod p$ from what they received and their own secret. Eve must solve the **discrete logarithm** (slide: $6^x \bmod 13 = 2$, so $x = 5$; $6^x \bmod 13 = 9$, $x = 4$), trivial for $p = 13$, infeasible for a 2048-bit p. DH yields a shared secret but *no authentication*: a **man in the middle** runs one exchange with Alice (key k_1) and another with Bob (key k_2) and relays, reading and altering everything ("Transfer \$10" → "Transfer \$100"). The endpoints must be authenticated, which is where Chapter 5 begins.

Hard questions, answered

Q. Which hard problem underlies DH and which underlies RSA?
A. Discrete logarithm; integer factorisation.

Q. Does encrypting the DH messages with Bob's public key stop the MITM?
A. Only if Alice already holds Bob's *authentic* public key; if Eve can substitute hers, she decrypts and relays. The issue is key authenticity, hence certificates.

Chapter takeaways

Summary: Symmetric cryptography

- Kerckhoffs: only the key is secret. Models COA < KPA < CPA/CCA; attack by analysis, brute force, people or implementation.
- Substitution falls to frequency analysis (Theorem 4.2.2); Vigenère splits into L Caesar groups; a non-repeating random key is the one-time pad (Theorem 4.3.1).
- Confusion (nonlinear S-boxes) and diffusion (permutation, mixing); Feistel inverts for free with reversed keys (Theorem 4.4.2).

- DES: 64/56, 16 rounds of E–XOR–S-box–P, rotations 1 in rounds 1, 2, 9, 16; brute-forced in 1998; 2DES falls to meet in the middle.
- AES: GF(2^8) with `1B`; SubBytes, ShiftRows, MixColumns, AddRoundKey; 10/12/14 rounds; SPN needs explicit inverses.
- ECB leaks structure; CBC chains with a public random IV; CTR is a parallel stream; CCM adds integrity.
- DH: discrete logarithm; shared secret without authentication, hence MITM.

Chapter 5

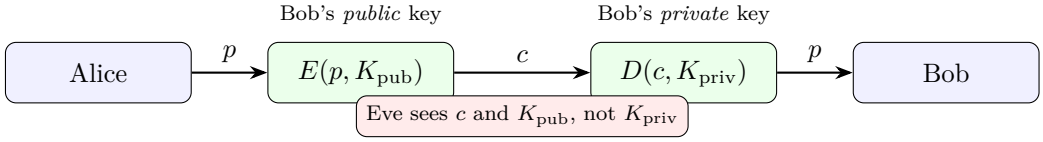
Public-Key Cryptography, Integrity and Trust

5.1 Why Asymmetric

What are we trying to get?

Symmetric cryptography needs a secret delivered in advance to every pair of parties. We want the idea that removes that requirement, the mathematical object it rests on, and the new problem it creates in exchange.

With one shared key per pair, N people need $\binom{N}{2} = N(N-1)/2$ keys (6 people: 15) and a secure channel for each. With key pairs, $2N$ keys (6 people: 12) and no pre-shared secret. The mailbox principle: anyone can drop a letter, only the owner can open the box. Diffie, Hellman and Merkle published the idea in 1976.



Definition 5.1.1 (One-way and trapdoor functions; requirements)**.** A **one-way function** $y = f(x)$ is easy to compute and infeasible to invert. A **trapdoor function** is one-way except for whoever holds secret information t, generated together with f by an algorithm Gen; the public key is f, the private key is t. A public-key system must make key generation, encryption with the public key and decryption with the private key *easy*, and deriving the private key from the public key or the message from the ciphertext *infeasible*. Candidate: integer factorisation. 6895601 is a product of two primes; finding them is slow, but given the trapdoor $t = 1931$ the division $6895601/1931 = 3571$ is immediate.

Key insight

Public keys remove the need for a *secret* channel and create the need for an *authentic* one: Alice must know that the public key is really Bob's, or a man in the middle substitutes his own (§5.5). Certificates (§5.6) exist for exactly this.

Hard questions, answered

Q. Why does asymmetric cryptography not eliminate the key-distribution problem but transform it?
A. Secrecy of the distributed key is no longer needed, authenticity is; a wrong public key gives Eve the plaintext.

Q. For $N = 100$ users, how many keys under each model?
A. Symmetric: $100 \cdot 99/2 = 4950$. Asymmetric: 200.

5.2 Number Theory for RSA

What are we trying to get?

RSA needs four facts about integers: congruences, inverses modulo n, Euler's totient, and Euler's theorem; plus fast exponentiation to compute with them.

Definition 5.2.1 (Modular arithmetic, inverses, totient)**.** $a \equiv b \pmod{n}$ iff $n \mid a - b$. An integer a has a multiplicative inverse modulo n iff $\gcd(a, n) = 1$; the extended Euclidean algorithm finds it (for tiny numbers, try $d = 1, 2, \ldots$ until $ed \bmod \varphi = 1$). **Euler's totient** $\varphi(n)$ counts the integers $1 \le a < n$ coprime to n; for distinct primes, $\varphi(pq) = (p-1)(q-1)$.

Theorem 5.2.2 (Euler; Fermat)**.** *If* $\gcd(a, n) = 1$ *then* $a^{\varphi(n)} \equiv 1 \pmod{n}$. *In particular (Fermat) for a prime* p *and* $p \nmid a$, $a^{p-1} \equiv 1 \pmod{p}$.

Proof. The map $x \mapsto ax$ permutes the $\varphi(n)$ residues coprime to n (it is injective because a is invertible). Multiplying all of them gives $\prod x \equiv a^{\varphi(n)} \prod x$, and $\prod x$ is invertible, so $a^{\varphi(n)} \equiv 1$. □

Technique: Fast exponentiation

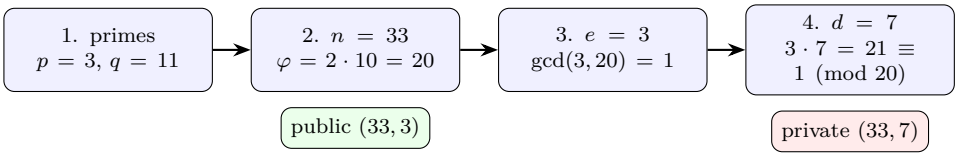
Square repeatedly and multiply the needed powers, reducing modulo n at every step: $26^7 \bmod 33$ via $26^2 \equiv 16$, $26^4 \equiv 16^2 = 256 \equiv 25$, $26^7 = 26^4 \cdot 26^2 \cdot 26 \equiv 25 \cdot 16 \cdot 26 \bmod 33 = 5$. Cost $O(\log e)$ multiplications instead of e.

5.3 RSA

What are we trying to get?

We want the algorithm, a proof that decryption undoes encryption, the worked numbers of the slides and the TP, and the three practical caveats (size of n, speed, determinism) that turn textbook RSA into usable RSA.

Definition 5.3.1 (RSA (Rivest, Shamir, Adleman, 1977))**.** Choose distinct large primes p, q (random, similar in size); $n = pq$; $\varphi = (p-1)(q-1)$; choose e with $1 < e < \varphi$ and $\gcd(e, \varphi) = 1$; compute d with $ed \equiv 1 \pmod{\varphi}$. Public key (n, e), private key (n, d); p, q, φ remain secret. Encrypt $c = m^e \bmod n$; decrypt $m = c^d \bmod n$.



Theorem 5.3.2 (Correctness of RSA)**.** *For every* $0 \leq m < n$, $(m^e)^d \equiv m \pmod{n}$.

Proof. $ed = 1 + k\varphi$ for some integer k. If $\gcd(m, n) = 1$, Theorem 5.2.2 gives $m^{ed} = m \cdot (m^\varphi)^k \equiv m$. If $p \mid m$, both sides are 0 modulo p; modulo q (which does not divide m) Fermat gives $m^{ed} = m \cdot (m^{q-1})^{k(p-1)} \equiv m$; the congruence holds modulo p and q, hence modulo n by the Chinese remainder theorem. The case $q \mid m$ is symmetric. □

Worked example: Slides and the TP password

Slide, $m = 4$: $c = 4^3 = 64 \bmod 33 = 31$; $31^7 \bmod 33 = 4$. Exercise, $m = 5$: $c = 125 \bmod 33 = 26$; $26^7 \bmod 33 = 5$ (technique above). TP, $p = 5$, $q = 11$, $e = 3$, $C = 23$: $n = 55$, $\varphi = 40$, $d = 27$ ($81 = 2 \cdot 40 + 1$). $23^2 = 529 \equiv 34$; $23^4 \equiv 34^2 = 1156 \equiv 1$; so $23^{27} = (23^4)^6 \cdot 23^3 \equiv 23^3 = 23 \cdot 34 = 782 \equiv 12$. Password $m = 12$; check $12^3 = 1728 = 31 \cdot 55 + 23 = C$. A fresh pair: $p = 7$, $q = 13$, $e = 5$: $n = 91$, $\varphi = 72$, $d = 29$; $m = 2 \mapsto c = 32$.

Interpretation & discussion

Security rests on factoring n: with p and q one computes φ and then d. The RSA Factoring Challenge marks the frontier: RSA-768 (232 digits) factored December 2009, RSA-250 (829 bits) February 2020; 1024-bit keys "are out there", 2048 bits is the default. A 50-bit modulus (TP) falls to trial division up to $\sqrt{n} \approx 2^{25} \approx 3.4 \times 10^7$ in seconds; a 512-bit modulus has $\sqrt{n} \approx 2^{256}$ and needs the number field sieve (deprecated, but not trial division). RSA is slow and the message must be smaller than n, so it encrypts a *symmetric key*, not a file: the **hybrid** scheme draws a random AES key and IV, encrypts the message under AES-256-CBC, and encrypts the AES key under the recipient's RSA public key. Textbook RSA is deterministic and malleable: a 4-digit PIN is found by encrypting all 10^4 candidates and comparing; real systems use randomised padding (OAEP).

From the TP: OpenSSL mission 3

`openssl genrsa -out k.pem 512` writes an *unencrypted* private key: whoever reads the file owns it. `openssl rsa -in k.pem -text -noout` prints n, e, d, p, q; `-pubout` extracts the public key, which may be stored in plaintext; `-aes256` at generation encrypts the key file under a passphrase with AES-256-CBC. `pkeyutl -encrypt -pubin` works on a short file and fails on a large one. Factoring the 50-bit `otherSmall_pub.pem`: extract n with `-pubin -text`, trial-divide to $\sqrt{n}$, compute d; the same loop on a 512-bit modulus never finishes.

Go deeper: the details an examiner asks about

- e must be coprime to φ or d does not exist; the common choice is $e = 65537 = 2^{16} + 1$.
- "Similar in magnitude": a small prime is found by trial division; primes too close fall to Fermat's method $n = a^2 - b^2$; $p = q$ makes n a perfect square and φ wrong.
- The slide says "modulo a prime"; precisely, modulo $n = pq$.

Hard questions, answered

Q. Why combine AES and RSA for a long message?
A. RSA is slow and bounded by n; AES is fast and unbounded. RSA protects only the 32-byte key, AES does the bulk, and no pre-shared secret is needed.

Q. Eve knows (n, e) and the ciphertext of a 4-digit PIN. Attack?
A. Encrypt all 10^4 PINs under (n, e) and compare: textbook RSA is deterministic. Fix: OAEP padding.

5.4 Hash Functions and Passwords

What are we trying to get?

A hash compresses any input to a fixed fingerprint. We want the three resistance properties and how they relate, the proof that a hash alone gives no authenticity, and the password-storage story (dictionaries, salts, slow hashes).

Definition 5.4.1 (Cryptographic hash)**.** H maps an arbitrary-length input to a fixed-length digest; it is fast, lossy (the input cannot be reconstructed), and a tiny change in the input produces a very different digest. Requirements: **preimage resistance** (given y, hard to find any x with $H(x) = y$), **second-preimage resistance** (given x, hard to find $x' \neq x$ with $H(x') = H(x)$), **collision resistance** (hard to find *any* pair $m_1 \neq m_2$ with equal digests). Collisions exist because the output space is finite; the question is the work to find one. Generic attacks: preimages cost about 2^n for an n-bit digest, collisions about $2^{n/2}$ (birthday bound). Collision resistance implies second-preimage resistance; neither implies preimage resistance. MD5 and SHA-1 have practical collisions; SHA-2 (224/256/384/512) is current.

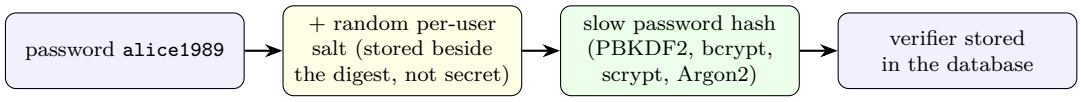
Theorem 5.4.2 (A hash alone gives no authenticity)**.** *If Alice sends $(m, H(m))$ over a channel Eve controls, Eve can replace it by $(m', H(m'))$ and Bob's check passes.*

Proof. H is public and takes no key; Eve computes $H(m')$ herself. Integrity against an adversary therefore needs a secret: a MAC key or a signing key (§5.5). A bare hash detects only *accidental* corruption. □

Theorem 5.4.3 (Birthday bound)**.** *Among k random n-bit digests the expected number of colliding pairs is $\approx k^2/2^{n+1}$, which reaches 1 at $k \approx 2^{n/2}$.*

Proof. There are $\binom{k}{2} \approx k^2/2$ pairs and each collides with probability 2^{-n}; by linearity of expectation the expected count is $k^2/2^{n+1}$. □

Password storage



Store H(password), never the password; at login hash the input and compare. A stolen digest is harder to use than plaintext, but *preimage resistance assumes an unknown high-entropy input*: a password drawn from a small dictionary is found by enumeration however good H is. **Dictionary attack**: precompute the digests of common passwords; unsalted, the same password gives the same digest for every user, so one table serves every account and equal digests reveal shared passwords. **Salt**: a random per-account value stored beside the digest and mixed into the hash; it need not be secret, because its purpose is to make each account's computation *different*: no shared table, one guessing campaign per user. Salts do not strengthen a weak password; *slow* password hashes raise the per-guess cost (OpenSSL's `-pbkdf2` is such a derivation).

Go deeper: the details an examiner asks about

- Hashing on the client before sending does not remove server-side hashing: the client-side hash *becomes* the password and can be replayed.
- "MD5 is broken" matters for signatures (an attacker crafting two documents with one digest, the 2008 rogue-CA case) more than for download checksums against accidental corruption, where a second preimage is still needed.

Hard questions, answered

Q. Two users share a password. Unsalted vs salted database?
A. Unsalted: identical digests; cracking one cracks both; precomputed tables apply to every account. Salted: different digests; separate work per account.

Q. Why is a 128-bit hash (MD5) considered too short for collisions even if the function were perfect?
A. By Theorem 5.4.3 a collision costs only 2^{64} work, within reach; preimages would still cost 2^{128}.

5.5 MACs and Digital Signatures

What are we trying to get?

Theorem 5.4.2 says integrity needs a secret. We want the two ways to add one, symmetric (MAC) and asymmetric (signature), what each guarantees and what each cannot, and why signatures are applied to a hash.



Definition 5.5.1 (MAC; digital signature)**.** A **message authentication code** uses a shared secret: Alice sends $(m, t = \mathrm{MAC}(K, m))$, Bob recomputes and compares; without K, Eve cannot produce a valid tag for a modified message. Constructions: the last block of CBC encryption of m (the sender needed key and message together; expensive), or hash-based schemes (hash the message and encrypt the digest symmetrically; hash the message concatenated with a secret value), which require a *cryptographic* hash. A MAC provides integrity and shared-key authentication, *no* confidentiality and *no* non-repudiation. A **digital signature** uses the sender's private key over the hash of the message and is verified with the sender's public key; it provides origin authentication, data integrity and signatory **non-repudiation** (NIST FIPS 186-4); algorithms RSA, DSA, ECDSA; no pre-shared key is needed.

Reading the formula

Why sign the hash. A 15 MB document under 1024-bit RSA would require thousands of slow operations, and signing raw messages of the attacker's choice exposes the private key to algebraic attacks; the digest is short and fixed-size, so the signature's size and cost are independent of the message. The scheme then needs collision resistance: if Eve finds m_1, m_2 with one digest and gets m_1 signed, the signature validates m_2. *Why no non-repudiation from a MAC.* Both parties hold K, so Bob could have produced the tag himself; a judge cannot tell who did. A signature uses a key only Alice holds.

	Confidentiality	Integrity	Authentication	Non-repudiation
Encryption	yes	no	no	no
MAC	no	yes	yes (shared key)	no
Signature	no	yes	yes	yes

Confidentiality and authenticity are separate: secrets sent anonymously (C without A), public information from a trusted source (A without C), secure communication (both). None of the three gives availability.

Hard questions, answered

Q. A confidential contract that Bob cannot later deny having received from Alice. Primitives and order?
A. Sign with Alice's private key over $H(m)$ (authenticity, non-repudiation), then encrypt for Bob (hybrid). Sign first so the signature binds the plaintext.

Q. Is $\mathrm{MAC} = H(K\|m)$ safe?
A. Not with Merkle–Damgård hashes (MD5, SHA-1, SHA-256): length extension lets Eve compute $H(K\|m\|\mathrm{pad}\|m')$ without K. HMAC (or SHA-3) fixes it.

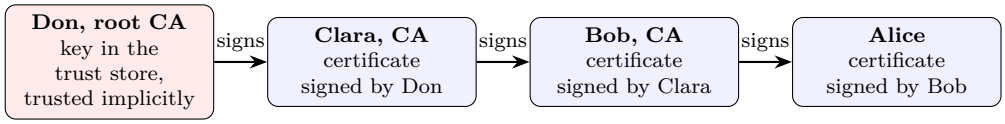
Q. Why can a MAC not serve as evidence in court?
A. The verifier holds the same key as the signer and could have produced the tag; a third party cannot attribute it. Only a signature attributes.

5.6 Certificates and PKI

What are we trying to get?

Signatures need the verifier to hold the signer's authentic public key, which is the problem that asymmetric cryptography created in §5.1. We want the recursion that solves it, where it bottoms out, and what depends on that base case.

To verify Alice's signature we need her public key; she cannot prove remotely that a key is hers by signing a message with that same key (verifying it would need the key in question). So someone else signs a message binding Alice's identity to her key; that signer's key is vouched for by another; and so on, until a key that "was shipped to us just for this" and is trusted implicitly.



to trust Alice's key, verify each signature from right to left until a root in the trust store

Definition 5.6.1 (Certificate, CA, chain of trust, CSR, PKI)**.** A **certificate** is a message containing an identity and a public key, signed by someone else; each signer is a **certificate authority** (CA); the implicitly trusted one is a **root CA**; the signatures form a **chain of trust**. Obtaining a certificate: (1) generate a key pair and keep the private key secret forever; (2) present the public key and identity to the CA in a **certificate signing request** (everything in the certificate except the signature) with proof of identity; (3) the CA verifies and signs. Format: X.509 (RFC 5280). **PKI** (RFC 4949): the set of hardware, software, people, policies and procedures needed to create, manage, store, distribute and *revoke* certificates; it includes a **trust store**, the list of root CAs and their keys, chosen by the OS or application vendor.

Interpretation & discussion

The base case is trust by *distribution*, not by cryptography: root keys arrive with the operating system. Whoever controls the trust store controls everything above it. Organisations add a private root to sign internal certificates; malware that adds a root can have it sign a certificate for *any* site and mount man-in-the-middle attacks on every HTTPS connection from that machine; some "security" appliances do exactly this to inspect encrypted traffic, which the course calls stupid and dangerous, since one compromised CA key breaks confidentiality for everyone behind it. What a browser checks: the server certificate's signature by the intermediate, the intermediate's by a root in the store, validity dates, that the name matches, revocation status; only then does it use the server's public key for key exchange.

Hard questions, answered

Q. Why can't Alice self-sign to prove her key is hers?
A. Verifying would need her public key, the thing in question; a self-signature shows only that the key pair's holder signed. Trust must enter from outside.

Q. CSR versus the CA generating the key pair: what changes?
A. With a CSR the private key never leaves Alice; the CA attests only the public key. If the CA generates the pair it holds the private key, which destroys non-repudiation and confidentiality guarantees.

Q. Malware adds a root CA to one laptop. What can the attacker do, and what stays safe?
A. Issue a certificate for any site that laptop's browser will accept, hence MITM every HTTPS connection from it (read and alter). Other machines are unaffected; certificate pinning or fingerprint checks may detect it.

5.7 Steganography

Cryptography hides the *meaning* of a message; steganography hides its *existence*. Simmons (1983) brought it into computer science as the *Prisoners' Problem*: Alice and Bob communicate through a warden; the carriers are *stego objects*, the hidden data the *payload*; a *passive* warden only observes (a detection problem for the warden), an *active* warden alters the objects (a robustness problem for Alice). The first recorded technique is from Greece around 440 BC. Null ciphers hide a message in the n-th letters of words (third letters of "Fishing freshwater bends and saltwater coasts rewards anyone feeling stressed…" spell "Send Lawyers, Guns, and Money"; first letters of the first paragraph and second letters of the second of the WWI telegram spell "PERSHING SAILS FROM N.Y. JUNE 1"). Types: text, image, audio, video, network and email steganography; the size of the Internet is what makes hiding effective. **LSB steganography** overwrites the least significant bit of each pixel or sample with payload bits: each value changes by at most 1/255, invisible; capacity one bit per byte; detectable by LSB statistics; destroyed by recompression. Encrypt the payload first so that detection does not reveal meaning.

Hard questions, answered

Q. Hide 'A' (`01000001`) in pixels `10110010, 10110011, 10110010, 10110011, 10110010, 10110011, 10110010, 10110011`.
A. Replace each LSB: `10110010, 10110011, 10110010, 10110010, 10110010, 10110010, 10110010, 10110011`; each value moves by at most one.

Q. Which warden defeats LSB trivially, and how?
A. The active warden: re-encode the image (JPEG, blur, resample); the LSB plane is destroyed while the picture survives for the receiver.

Chapter takeaways

Summary: Public-key cryptography, integrity, trust

- $N(N-1)/2$ vs $2N$ keys; trapdoor one-way functions; the authenticity of public keys becomes the problem.
- RSA: $n = pq$, φ, $ed \equiv 1$; correct by Euler (Theorem 5.3.2); secure by factoring; hybrid with AES; padding against determinism.
- Hash: preimage, second preimage, collision ($2^{n/2}$, Theorem 5.4.3); keyless, so no authenticity (Theorem 5.4.2); passwords: salt against shared tables, slow hashes against speed.
- MAC (shared key: integrity and authentication) vs signature (private key over $H(m)$: plus non-repudiation); encryption gives confidentiality only.
- Certificates bind identity to key by a CA's signature; chains end at a distributed root; the trust store is the root of everything; PKI includes revocation.
- Steganography hides existence; LSB; passive vs active warden.